

Appunti di Algoritmi Distribuiti

Questa è la versione pubblicata con MkDocs, con navigazione ad albero e build automatica su GitHub Pages.

Capitoli pubblicati

- 0. Ripasso sugli algoritmi
- 1. Teoria della complessità
- 4. Fondamenti di algoritmi distribuiti
- 5. Spanning Tree Construction

PDF unico

Il PDF completo viene generato in CI a ogni push su `main` ed è scaricabile dagli artifact del workflow: `pages.yml`

Workflow locale docs

```
just sync
just browser # una tantum
just build
just fix-images
# opzionale: mantiene i riferimenti assets cosi' come sono
# just fix-images-no-normalize
# pipeline completa (sync + browser + build + pdf in dist/)
# just docs
```

Ripasso

Problema Computazionale

Un **problema computazionale** Π è una questione generale che dipende da parametri i cui valori non sono specificati. Si definisce tramite:

- **INPUT:** insieme I di possibili assegnamenti ai parametri
- **OUTPUT:** insieme S di possibili soluzioni
- **Funzione:** $\Pi : I \rightarrow S$

Un'istanza del problema si ottiene assegnando valori specifici ai parametri. *Per esempio, definisco il problema "Ordinare una sequenza di numeri" e un'istanza del problema può essere "Ordinare la sequenza {5, 2, 9, 1}"*.

Classificazione dei Problemi per Tipo di Soluzione

I problemi si classificano in base al tipo di soluzione cercata:

1. **Problemi Decisionali:** la risposta è binaria (**SÌ/NO**)
 - *Esempio:* "Il numero x è primo?" o "Esiste un cammino da A a B di lunghezza $\leq K$?"
 2. **Problemi di Ricerca:** cerchiamo una soluzione che soddisfi certe proprietà.
 - *Esempio:* "Trovami un cammino da A a B".
 3. **Problemi di Ottimizzazione:** Cerchiamo la soluzione **migliore** (minimo costo o massimo guadagno) tra tutte quelle ammissibili.
 - *Esempio:* "Trovami il cammino *più breve* da A a B".
-

Algoritmi

Un **algoritmo** è una procedura generale per risolvere un problema definita tramite una **sequenza di passi finita, ben ordinata, non ambigua**, effettivamente **realizzabile** e che **termina** in tempo finito.

Un algoritmo per il problema Π è **corretto** se, per ogni istanza $i \in I$, produce la soluzione corrispondente $\Pi(i) \in S$.

Due proprietà chiave: - **Correttezza:** L'algoritmo deve terminare e produrre l'output corretto per *ogni* istanza valida - **Efficienza:** Misuriamo quanto "costa" l'algoritmo in termini di risorse (indipendentemente dalla macchina). - Tempo: Numero di operazioni - Spazio: Numero di celle di memoria

Dimensione del Problema

La dimensione misura la quantità di informazione necessaria per specificare l'istanza:

- **Criterio di costo logaritmico:** numero di bit necessari per rappresentare l'input
- **Criterio di costo uniforme:** numero di "elementi" necessari per rappresentare l'input

Analisi Asintotica

Non misuriamo il **tempo** in secondi, ma in **numero di operazioni elementari** in funzione della dimensione dell'input (n).

- **Caso Peggior (Worst Case):** Il tempo massimo richiesto per un input di dimensione n . È la garanzia che diamo.
- **Notazioni:**
 - $O(f(n))$: Limite superiore (non va peggio di così).
 - $\Omega(f(n))$: Limite inferiore (serve almeno questo tempo).
 - $\Theta(f(n))$: Ordine di grandezza esatto (limite superiore e inferiore coincidono).

Notazione "O-grande" $O(\cdot)$



Espressione O	nome
$O(1)$	(sublineare) costante
$O(\log \log n)$	(sublineare) log log
$O(\log n)$	(sublineare) logaritmico
$O(\sqrt[n]{n}), c > 1$	sublineare
$O(n)$	lineare
$O(n \log n)$	$n \log n$
$O(n^2)$	quadratico
$O(n^3)$	cubico
$O(n^k) (k \geq 1)$	polinomiale
$O(a^n) (a > 1)$	esponenziale
$O(n!)$	fattoriale

INTERPRETAZIONE

Una funzione $f(n)$ in **ALTO** e' **O-grande** di una qualsiasi funzione $g(n)$ più in **BASSO**

ESEMPI

$$\sqrt[2]{n} \in O(n)$$

$$n^{10} \in O(2^n)$$

$$7^n \in O(n!)$$

■ perfetto ■ buono ■ accettabile ■ inaccettabile

Classificazione dei Problemi per Difficoltà

In base al costo computazionale, i problemi si classificano in:

1. **Problemi Trattabili (facili, classe P):** Esiste un algoritmo che li risolve in tempo **polinomiale** (es. $O(n)$, $O(n^2)$, $O(n \log n)$)

$$T(n) \in O(n^k), k \geq 0$$

- *Esempi:* Ordinamento, Cammino Minimo (Dijkstra), Spanning Tree.
 - *Perché:* Se raddoppio l'input, il tempo aumenta di un fattore gestibile.
2. **Problemi Presumibilmente Intrattabili (difficili):** non abbiamo un algoritmo di costo polinomiale, ma non è stato dimostrato che non esista
 3. **Problemi Intrattabili:** si può dimostrare che non esiste un algoritmo di costo polinomiale. Il costo è **esponenziale** (es. $O(2^n)$, $O(n!)$).
 - *Esempi:* Tutti i possibili sottoinsiemi, tutte le permutazioni (TSP forza bruta), Torre di Hanoi.
 - *Perché:* Basta aggiungere un piccolo elemento all'input (es. una città in più nel TSP) e il tempo di calcolo raddoppia o peggio. Diventa inutilizzabile molto presto.
 4. **Problemi Irrisolvibili:** si può dimostrare che non esiste alcun algoritmo risolutivo (indipendentemente dal costo)
 - Esempio: **Problema della fermata** (dato un algoritmo A con input D, l'esecuzione di A con input D termina in tempo finito?)
-

Perché il Polinomiale è la Soglia di Trattabilità?

I problemi trattabili sono quelli di costo polinomiale perché:

- Esistono pochi problemi trattabili con algoritmi polinomiali di grado alto
 - In molti problemi, lo spazio delle soluzioni è esponenziale: trovare un algoritmo polinomiale significa "fare meglio" di un algoritmo di forza bruta
 - Le istanze worst case sono spesso rare o poco probabili
 - Costi peggiori del polinomiale diventano rapidamente intrattabili al crescere della dimensione delle istanze
 - È una misura indipendente dal progresso tecnologico
-

DOMANDA: l'appartenenza dei problemi a queste classi di problemi, non potrebbe dipendere dal modello di calcolo?

Tesi di Church-Turing

Modelli di calcolo diversi, ma ragionevoli, si possono simulare a vicenda con uno **slowdown polinomiale**. Sono tutti polinomialmente equivalenti alla Macchina di Turing.

Le classi di problemi non cambiano cambiando modello di calcolo, purché quest'ultimo sia ragionevole.

Riassunto - Teoria della Complessità

La teoria della complessità classifica i problemi in base alla difficoltà di risolverli efficientemente. Si concentra sui **problemi decisionali** (output SI o NO), poiché la trattazione formale è più agevole e permette di trarre conclusioni applicabili anche ad altri tipi di problemi.

Esempio: Il problema **Shortest Path** può essere espresso sia come problema di ottimizzazione (qual è la lunghezza del cammino minimo?) che come problema decisionale (esiste un cammino di lunghezza $\leq k$?).

Classificazione dei problemi

Classe P (Polynomial Time) - "I Facili"

Sono i problemi che un computer può **RISOLVERE** velocemente (in tempo polinomiale, $O(n^k)$).

- **Esempio:** Ordinare una lista (Sorting), trovare il cammino minimo (Dijkstra).
- **Nota:** Se un problema è in P, è automaticamente anche in NP.

Classe NP (Nondeterministic Polynomial Time) - "I Verificabili"

Sono i problemi per cui, se io ti do una soluzione ("il certificato"), tu puoi **VERIFICARE** velocemente se è corretta con un algoritmo deterministico in tempo polinomiale.

- *Definizioni utili*
 - **Certificato:** sequenza di caratteri di dimensione al massimo polinomiale che contiene l'evidenza che un'istanza i sia positiva per il problema Π .
 - **Algoritmo verificatore:** algoritmo decisionale che prende in input un'istanza i e un certificato C_i , restituendo SI se i è positiva per Π , NO altrimenti.
- **Esempio:** Fare un Sudoku è difficile (ci metti tempo a risolverlo), ma se ti do un Sudoku già compilato, ci metti un attimo a controllare se le regole sono rispettate.
- **Relazione:** $P \subseteq NP$. (Se so risolvere un problema velocemente, so anche verificare la soluzione velocemente).

Classe NP-Hard - "I Più Cattivi di Tutti"

Qui usciamo dal recinto dei problemi "trattabili". Un problema è NP-Hard se è **almeno** difficile quanto il problema più difficile in NP.

- **La Chiave (Riduzione):** Un problema H è NP-Hard se **ogni** problema in NP può essere ridotto (trasformato) in H in tempo polinomiale.

- **Significato:** Se trovassi un modo veloce per risolvere un problema NP-Hard, avrei trovato un modo veloce per risolvere *tutti* i problemi in NP (e quindi avrei dimostrato che $P=NP$).
- **Nota:** Un problema NP-Hard **non deve per forza essere in NP**. Potrebbe essere così difficile che non riesci nemmeno a verificare la soluzione velocemente (o potrebbe essere indecidibile, come l'Halting Problem).

Classe NP-Complete - "L'Intersezione Critica"

Questi sono i problemi "chiave". Sono sia in NP che NP-Hard.

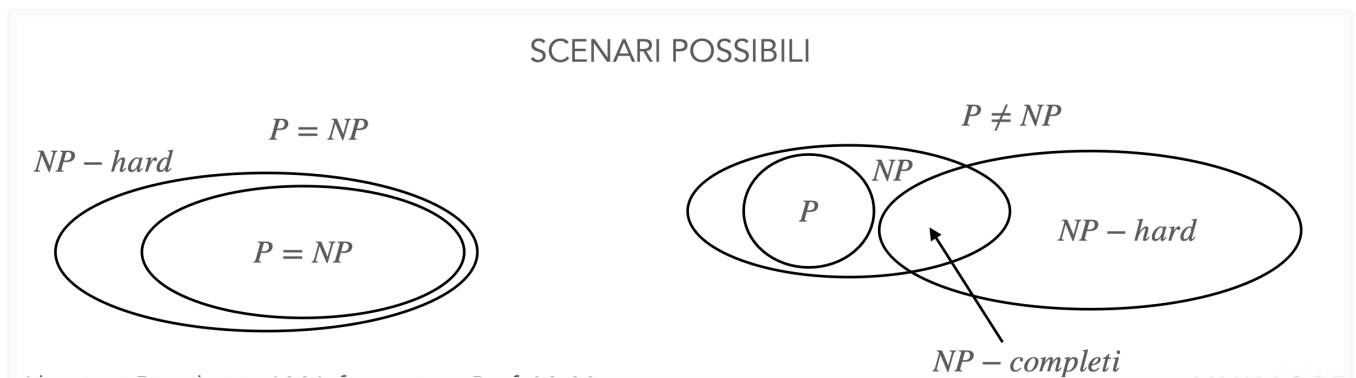
- **Definizione:** Un problema C è NP-Complete se soddisfa due condizioni:
 1. $C \in NP$ (è verificabile efficientemente).
 2. $C \in NP\text{-Hard}$ (tutti i problemi in NP si riducono a lui).
- **Concetto Chiave:** Rappresentano il "nucleo" della difficoltà di NP.
- **Esempi:** Vertex Cover, 3-SAT, TSP Decisionale.

Domanda: Il problema del commesso viaggiatore (TSP) è NP-Complete?

- **TSP Decisionale:** "Esiste un ciclo di costo minore di k ?" → **NP-Complete**. (Posso verificare se un ciclo dato costa meno di k).
- **TSP di Ottimizzazione:** "Trovami il ciclo di costo minimo assoluto." → **NP-Hard**, ma **non** NP-Complete.
 - *Perché?* Se ti do una soluzione e ti dico "Questa è la minima", tu non hai un modo veloce per verificare che non ne esista una ancora più piccola da qualche parte. Devi fidarti. Quindi non è in NP (come definito classicamente per i problemi decisionali).

Come rappresentiamo queste classi di problemi mantenendo il dubbio che in futuro si possano trovare degli algoritmi polinomiali risolutori dei problemi NP?

A sinistra siamo nel caso in cui i problemi NP potrebbero essere risolti in tempo polinomiale, a destra nel caso sia stato dimostrato che non esiste un algoritmo polinomiale per i problemi NP.



Classe	Trovare Soluzione (Tempo Polinomiale)	Verificare Soluzione (Tempo Polinomiale)	Relazione Insiemistica
P	SÌ	SÌ	$P \subseteq NP$
NP	NO (o non lo sappiamo)	SÌ	Contiene P e NP-Complete
NP-Complete	NO	SÌ	Intersezione tra NP e NP-Hard
NP-Hard	NO	NO (non garantito)	Almeno difficile quanto ogni problema in NP

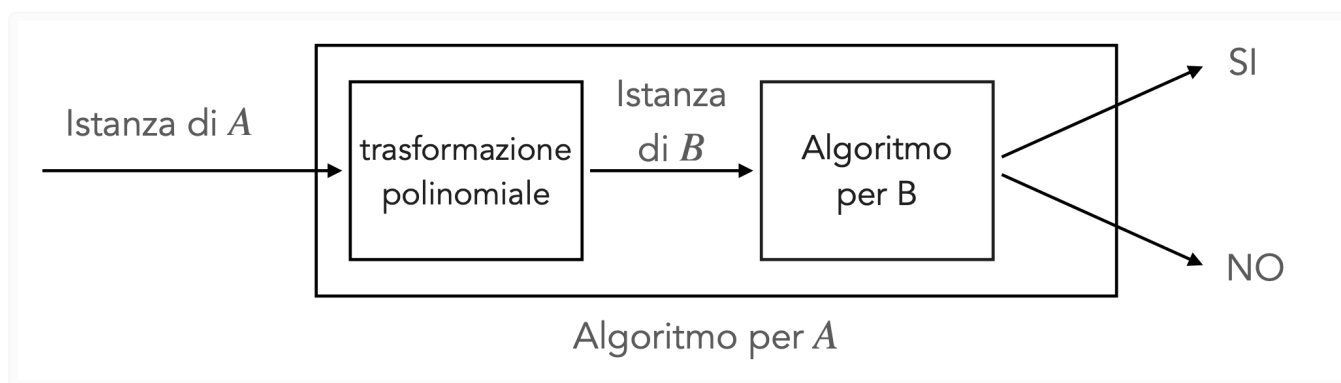
Confrontiamo la difficoltà di due algoritmi: Riduzione di Karp

Definizione: Un problema decisionale A è **riducibile in tempo polinomiale** a B ($A \leq_p B$) se:

- Ogni istanza di A può essere trasformata in tempo polinomiale in un'istanza di B
- Ogni istanza positiva di A viene trasformata in un'istanza positiva di B
- Ogni istanza negativa di A viene trasformata in un'istanza negativa di B

Intuizione: $A \leq_p B$ significa che:

- Il problema B non è più facile di A (un algoritmo per B può risolvere A)
- Il problema A non è più difficile di B

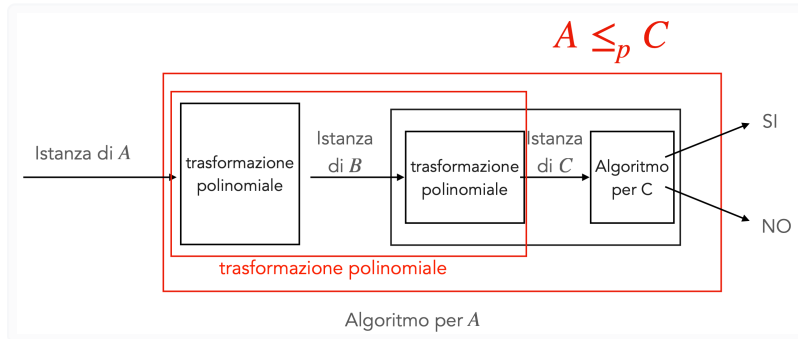
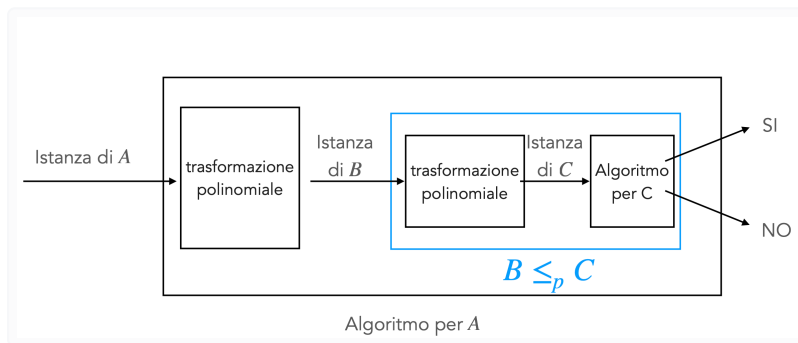
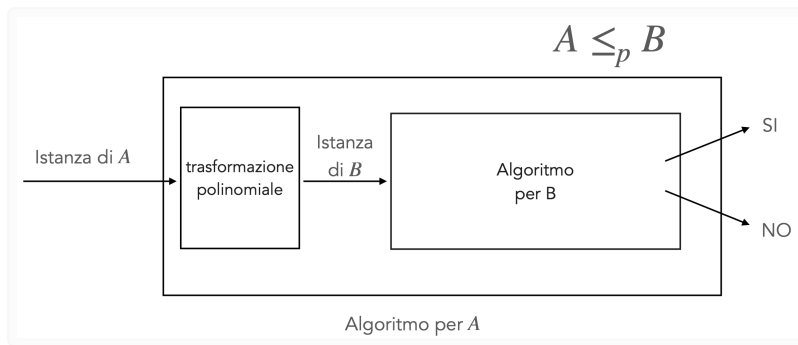


Conseguenze

Se $A \leq_p B$ allora:

- $B \in P \Rightarrow A \in P$ 🚀 abbiamo un algoritmo polinomiale per A 🚀
- $A \notin P \Rightarrow B \notin P$ non abbiamo un algoritmo polinomiale per A 😞, quindi neanche per B 😞
- $B \leq_p A \Rightarrow A \equiv B$ i problemi sono equivalentemente difficili

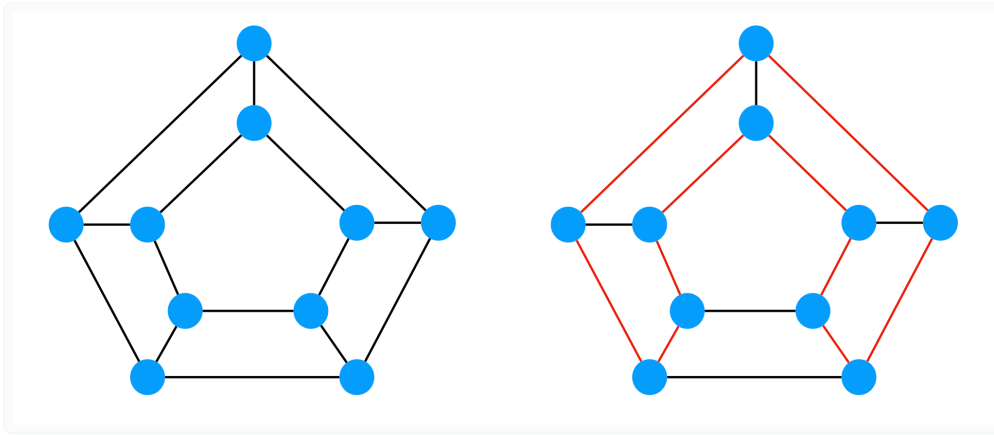
Proprietà: La riduzione polinomiale è **transitiva**: $A \leq_p B \wedge B \leq_p C \Rightarrow A \leq_p C$.



Classe NP (Non-deterministic Polynomial Time)

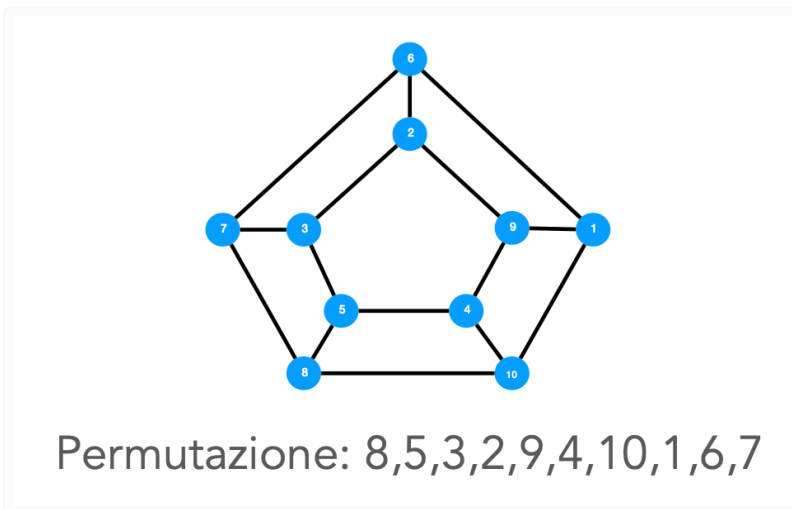
Esempio: Ciclo Hamiltoniano

- **INPUT:** Grafo $G = (V, E)$
- **OUTPUT:** Esiste un ciclo in G che passa per tutti i nodi esattamente una volta?



Per questo problema:

- Non si conoscono algoritmi polinomiali
- Un algoritmo forza bruta richiede tempo **fattoriale**
- Possiamo **verificare** una soluzione in tempo polinomiale: data una permutazione dei nodi, possiamo verificare in tempo lineare se rappresenta un ciclo hamiltoniano



Classe NP-Completo

Definizione: Un problema decisionale A è **NP-completo** se:

1. $A \in NP$
2. $\forall B \in NP : B \leq_p A$

Come dimostrare che un Problema è NP-Completo

Per dimostrare che A è NP-completo, sfruttiamo la transitività della riduzione:

1. Dimostrare che $A \in NP$
2. Scegliere un problema B già dimostrato NP-completo
3. Dimostrare che $B \leq_p A$

Teoremi di Cook (1971) e Levin (1973): Il primo problema dimostrato NP-completo è **SAT** (Satisfiability).

SAT: Data una formula in forma normale congiuntiva (FNC), esiste un assegnamento di verità che la renda vera?

- Esempio: $(a \vee b \vee c) \wedge (\neg b \vee c) \wedge (\neg a \vee \neg b \vee c)$

P.S. and $-->$ $\wedge$ or $-->$ $\vee$

Teorema Fondamentale

Se si dimostra che un problema NP-completo:

1. Ha un algoritmo polinomiale, allora $P = NP$
 2. Non ha algoritmi polinomiali, allora $P \neq NP$
-

Classe NP-Hard

Definizione: Un problema A (non necessariamente decisionale) è **NP-hard** se:

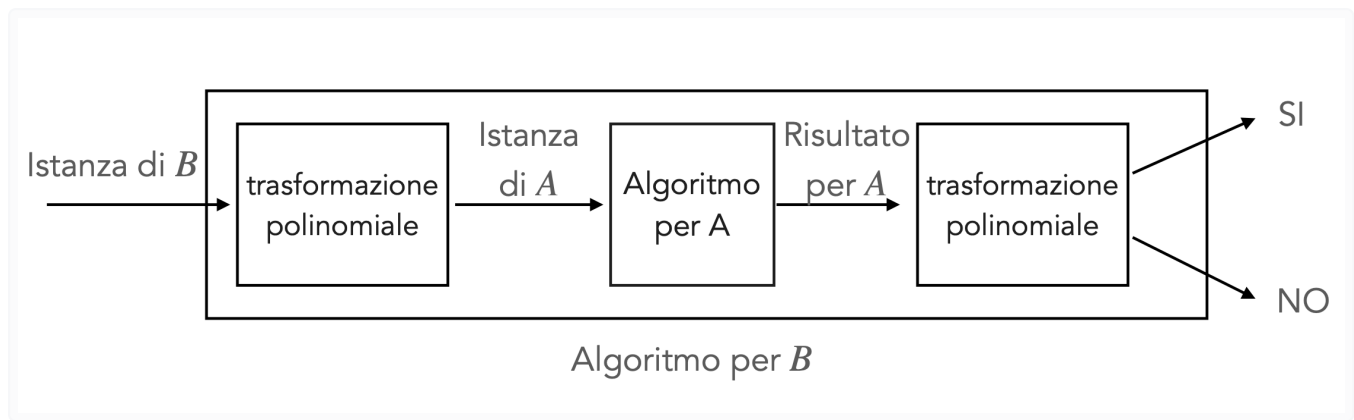
- $\forall B \in NP : B \leq_p A$

Differenza con NP-completo: un problema NP-hard non è necessariamente in NP (ad esempio, problemi di ottimizzazione).

Come Dimostrare che un Problema è NP-Hard

Per dimostrare che A è NP-hard:

1. Scegliere un problema B già dimostrato NP-completo
2. Dimostrare che $B \leq_p A$



N.B.: Non è detto che A abbia un algoritmo verificatore polinomiale

Soluzioni per Problemi NP-Hard

Molti problemi NP-hard hanno importanti applicazioni e non possono essere ignorati. Possibili soluzioni:

- Risolvere **efficientemente solo istanze piccole**
- Utilizzare **euristiche**
- Utilizzare algoritmi **paralleli/distribuiti**
- Utilizzare **algoritmi di approssimazione**: restituiscono in tempo polinomiale una soluzione ammissibile con garanzie sul discostamento dal costo ottimo

Teorema: Il problema del Commesso Viaggiatore (TSP) è NP-Hard

Obiettivo: Dimostrare che $TSP(Optimization) \geq_p HC(decisionale)$.

Vogliamo dimostrare che se avessimo un oracolo (un algoritmo efficiente) in grado di risolvere il TSP, potremmo usarlo per risolvere il problema del Ciclo Hamiltoniano (HC).

Poiché il problema del Ciclo Hamiltoniano (HC) è noto essere NP-Completo, se dimostriamo che esso è riducibile polinomialmente al TSP di ottimizzazione, allora il TSP è almeno tanto difficile quanto un problema NP-Completo, dunque è NP-Hard.

1. Intuizione e Strategia della Riduzione

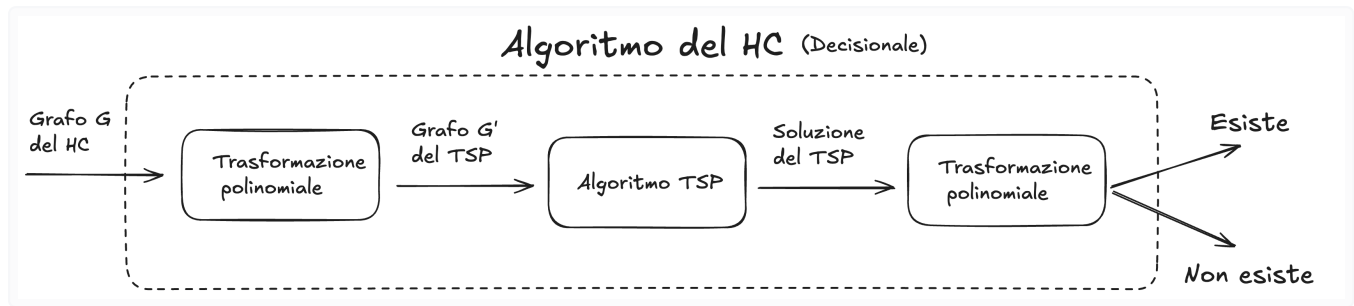
Il problema HC è un problema decisionale (Esiste un ciclo? Sì/No), mentre il TSP è un problema di ottimizzazione (Trova il ciclo di costo minimo).

Per collegarli, dobbiamo trasformare la struttura topologica del grafo di HC in costi per il TSP.

L'idea chiave è penalizzare l'uso di archi non esistenti:

- Assegniamo costo **0** agli archi che esistono realmente (nessuna penalità).
- Assegniamo costo **1** agli archi che non esistono (penalità).

In questo modo, il costo totale del tour fungerà da "rivelatore": se il costo è 0, il tour non ha mai "barato" (ha usato solo archi esistenti); se il costo è ≥ 1 , il tour è stato costretto a "barare" (ha usato archi che non c'erano in origine).



2. Definizione Formale della Riduzione

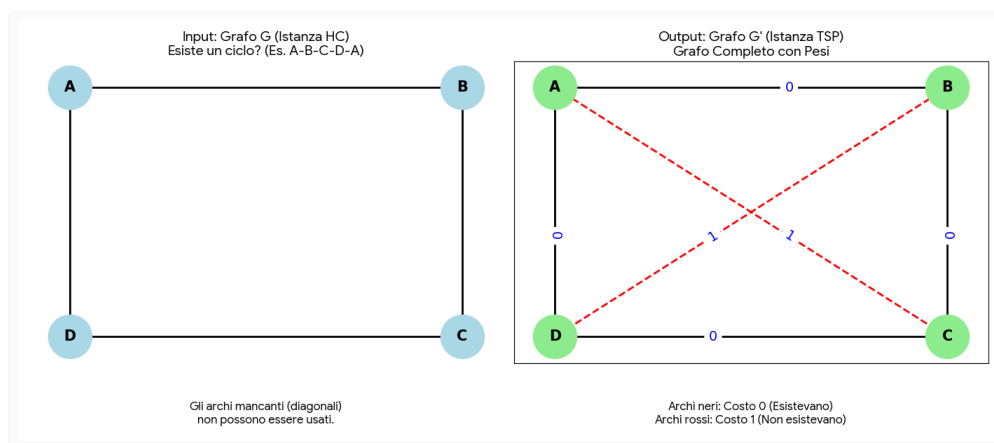
Costruiamo un algoritmo di trasformazione f che mappa un'istanza G di HC in un'istanza G' di TSP. - **Input:** Grafo $G = (V, E)$. - **Output:** Grafo completo ponderato $G' = (V, E')$ con funzione costo c .

Costruzione di G' :

1. **Vertici:** $V' = V$ (stessi nodi).
2. **Archivi:** $E' = V \times V$ (grafo completo: colleghiamo tutto con tutto).
3. **Pesi:** Per ogni coppia (u, v) , il peso è definito come:

$$c(u, v) = \begin{cases} 0 & \text{se } (u, v) \in E \text{ (arco originale)} \\ 1 & \text{se } (u, v) \notin E \text{ (arco fittizio)} \end{cases}$$

Nota: La costruzione di G' richiede di iterare su tutte le coppie di nodi, impiegando un tempo $O(|V|^2)$, che è polinomiale.



3. Dimostrazione di Correttezza

Sia C_{min} il valore della soluzione ottima restituita dal TSP su G' . Dobbiamo dimostrare che $C_{min} = 0 \iff G$ ha un HC.

Direzione A

Ipotesi: G ha un HC $\implies$ **Tesi:** $C_{min} = 0$

- **Ragionamento:** Se G ha un ciclo Hamiltoniano, esiste una permutazione di vertici collegati solo da archi in E .
- **Nel grafo TSP:** Poiché abbiamo mappato tutti gli archi di E con peso 0, questo ciclo esiste anche in G' e ha costo somma 0. Dato che i pesi sono non-negativi, 0 è il minimo possibile.
- **Conclusione:** L'algoritmo TSP troverà questo tour (o uno equivalente) e restituirà 0.

Direzione B

Ipotesi: $C_{min} = 0 \implies$ **Tesi:** G ha un HC

(Utilizziamo l'implicazione diretta sul valore del costo, che è più intuitiva della contronominale per chi legge).

- **Ragionamento:** Supponiamo che l'algoritmo TSP restituisca un tour di costo 0.
- **Analisi dei pesi:** Poiché la somma dei pesi è 0 e i pesi possibili sono solo 0, 1, questo implica che **ogni singolo arco** del tour ha peso 0.
- **Legame con G :** Per costruzione, un arco ha peso 0 in G' solo se esisteva in G .
- **Conclusione:** Il tour trovato dal TSP è quindi composto esclusivamente da archi originali di G . Poiché un tour TSP visita ogni nodo esattamente una volta, questo corrisponde esattamente alla definizione di Ciclo Hamiltoniano in G .

Nota logica Dimostrando $C_{min} = 0 \implies G$ ha un HC abbiamo dimostrato anche la sua contronominale:

Ipotesi: G non ha un HC $\implies$ **Tesi:** $C_{min} \neq 0$ (C non può essere negativa, quindi $C \geq 1$)

Infatti, se $C_{min} \geq 1$, significa che ogni possibile tour deve usare almeno un arco fittizio, quindi non esiste un HC in G .

4. Conclusione

La riduzione è polinomiale e valida. Risolvere il TSP permette di decidere HC:

- Se $TSP(G') = 0 \rightarrow$ Risposta HC: **SÌ**.
- Se $TSP(G') \geq 1 \rightarrow$ Risposta HC: **NO**.

Poiché HC è NP-Completo, **TSP (Ottimizzazione) è NP-Hard**.

Approfondimento: Perché scegliere pesi 1 e 2 invece di 0 e 1?

La scelta dei pesi nella riduzione non è casuale, ma determina **quale variante** del TSP stiamo dimostrando essere NP-Hard.

1. Il Test della Disuguaglianza Triangolare

La disuguaglianza triangolare è la regola che rende un grafo "geometricamente sensato". Essa afferma che andare direttamente da A a B non deve mai costare più che passare attraverso un intermedio C :

$$c(u, w) \leq c(u, v) + c(v, w)$$

Analizziamo i due casi:

Caso A: Pesi 0, 1 (La riduzione "Base")

- Arco presente (u, v) e $(v, w) \rightarrow$ costo 0.
- Arco mancante $(u, w) \rightarrow$ costo 1.
- **Verifica:** $1 \leq 0 + 0 \implies 1 \leq 0$ (**FALSO!**)
- **Conclusione:** Questa riduzione genera un grafo che viola la geometria euclidea. Stiamo dimostrando la difficoltà del **TSP Generale**.

Caso B: Pesi 1, 2 (La riduzione "Metrica")

- Arco presente (u, v) e $(v, w) \rightarrow$ costo 1.
- Arco mancante $(u, w) \rightarrow$ costo 2.
- **Verifica:** $2 \leq 1 + 1 \implies 2 \leq 2$ (**VERO!**)
- **Conclusione:** Questa riduzione genera un grafo che rispetta la geometria. Stiamo dimostrando la difficoltà del **Metric-TSP**.

2. Perché questa distinzione è cruciale? (Il vero motivo teorico)

La distinzione riguarda l'**approssimabilità**:

1. TSP Generale (Pesi 0/1):

È un problema "cattivo". Non solo è NP-Hard, ma non è nemmeno approssimabile.

Teorema: Se esistesse un algoritmo che approssima il TSP generale entro un fattore polinomiale, allora $P = NP$.

Quindi, con pesi 0/1 dimostriamo che il caso peggiore assoluto è intrattabile.

2. Metric-TSP (Pesi 1/2):

È un problema "più gentile". È NP-Hard (come dimostrato dalla riduzione con pesi 1/2), ma appartiene alla classe APX.

Esistono algoritmi efficienti che garantiscono una soluzione vicina all'ottimo:

- **2-Approximation:** Basato sull'Albero di Copertura Minimo (MST).
 - **Algoritmo di Christofides:** Garantisce un'approssimazione di fattore 1.5 (o $3/2$).
-

Fattore di approssimazione (questa parte devo ancora capirla)

Il **fattore di approssimazione** (spesso indicato con la lettera greca ρ , "rho", o α) è la misura della "garanzia di qualità" che un algoritmo ci offre nel caso peggiore.

Definizione Formale di Algoritmo di Approssimazione

Sia Π un problema di ottimizzazione (minimizzazione o massimizzazione) e sia I una generica istanza di questo problema.

Indichiamo con:

- $OPT(I)$: il valore della soluzione ottima per l'istanza I .
- $A(I)$: il valore della soluzione restituita dal nostro algoritmo (deterministico) A per l'istanza I .

Diciamo che l'algoritmo A è un **algoritmo di ρ -approssimazione** (o ha un fattore di approssimazione ρ) se, per **ogni** istanza I di dimensione n , vale la seguente relazione:

$$A(I) \leq \rho \cdot OPT(I) \Rightarrow \frac{A(I)}{OPT(I)} \leq \rho(n) \quad (\text{per problemi di minimizzazione})$$

oppure

$$A(I) \geq \rho \cdot OPT(I) \Rightarrow \frac{OPT(I)}{A(I)} \leq \rho(n) \quad (\text{per problemi di massimizzazione})$$

In entrambi i casi, il rapporto è definito in modo da essere sempre ≥ 1 .

- Se $\rho(n) = 1$, l'algoritmo è esatto (trova l'ottimo).
- Se $\rho(n) > 1$, l'algoritmo è approssimato. Più ρ è vicino a 1, migliore è l'algoritmo.

Nota Bene:

Il fattore ρ è una garanzia nel caso peggiore (worst-case).

Non stiamo dicendo che l'algoritmo sbaglia sempre di un fattore ρ . Stiamo dicendo che non sbaglierà mai più di un fattore ρ , nemmeno nell'istanza più "cattiva" possibile che l'esaminatore possa inventare.

Tipologie di Fattori di Approssimazione

Nel nostro corso, incontriamo principalmente tre classi di ρ :

1. Fattore Costante ($\rho = k$):

Il caso ideale per un problema NP-Hard. L'errore non cresce con la dimensione dell'input.

- *Esempio:* **Vertex Cover** ha $\rho = 2$.
- *Esempio:* **Metric TSP** (con disuguaglianza triangolare) ha $\rho = 2$ (o $\rho = 1.5$ con Christofides).

2. Fattore Logaritmico ($\rho = O(\log n)$):

L'errore cresce lentamente al crescere dell'input.

- *Esempio classico: Set Cover* (Copertura di Insiemi). Non puoi avere un fattore costante qui (salvo $P=NP$), il meglio che possiamo fare è $\ln n$.

3. Fattore Polinomiale ($\rho = O(n^k)$):

L'approssimazione degrada molto rapidamente. Spesso è inutile in pratica, ma teoricamente interessante.

- *Esempio: General TSP* (senza disuguaglianza triangolare). Qui il fattore di approssimazione può essere arbitrariamente grande (2^n , o peggio), rendendo il problema *inapprossimabile*.
-

La Tecnica Formale di Dimostrazione (Il "Lower Bound")

Come facciamo a calcolare ρ se non conosciamo $OPT(I)$?

Formalizziamo il ragionamento che abbiamo fatto prima.

Per dimostrare che un algoritmo A è una ρ -approssimazione per un problema di **minimizzazione**, dobbiamo trovare un **Lower Bound (LB)** tale che:

$$LB(I) \leq OPT(I)$$

E dimostrare che il nostro algoritmo soddisfa:

$$A(I) \leq \rho \cdot LB(I)$$

Combinando le due disuguaglianze:

$$A(I) \leq \underbrace{\rho \cdot LB(I)}_{\text{Limite calcolabile}} \leq \underbrace{\rho \cdot OPT(I)}$$

$$\Rightarrow \frac{A(I)}{OPT(I)} \leq \rho$$

Q.E.D. (Come Volevasi Dimostrare).

Esempio: Il Vertex Cover

Applichiamo il rigore al Vertex Cover.

1. Algoritmo A :

- Trova un *Matching Massimale* M nel grafo $G = (V, E)$.
- Restituisci l'insieme S composto da *tutti* gli estremi degli archi in M .
- Costo: $|S| = 2 \cdot |M|$.

2. Il Lower Bound (LB):

- Sia S^* la soluzione ottima (il Vertex Cover minimo).

- Poiché gli archi in M sono disgiunti (non condividono vertici), per coprirli tutti S^* deve contenere almeno un vertice per ogni arco in M .
- Formalmente: $|S^*| \geq |M|$. Questo è il nostro *LB*.

3. Calcolo di ρ :

$$\frac{A(I)}{OPT(I)} = \frac{|S|}{|S^*|} \leq \frac{2 \cdot |M|}{|M|} = 2$$

Quindi, l'algoritmo è una **2-approssimazione**.

Vertex cover

Dato un grafo non orientato $G = (V, E)$:

- Un **Vertex Cover** è un sottoinsieme di vertici $V' \subseteq V$ tale che per ogni arco $(u, v) \in E$, si ha che $u \in V'$ oppure $v \in V'$ (o entrambi).
- **Problema di Ottimizzazione**: Trovare il V' con la cardinalità minima (minimo numero di vertici).
- **Problema Decisionale**: Dato G e un intero k , esiste un Vertex Cover di dimensione $\leq k$?

La Complessità (Perché è "difficile"?)

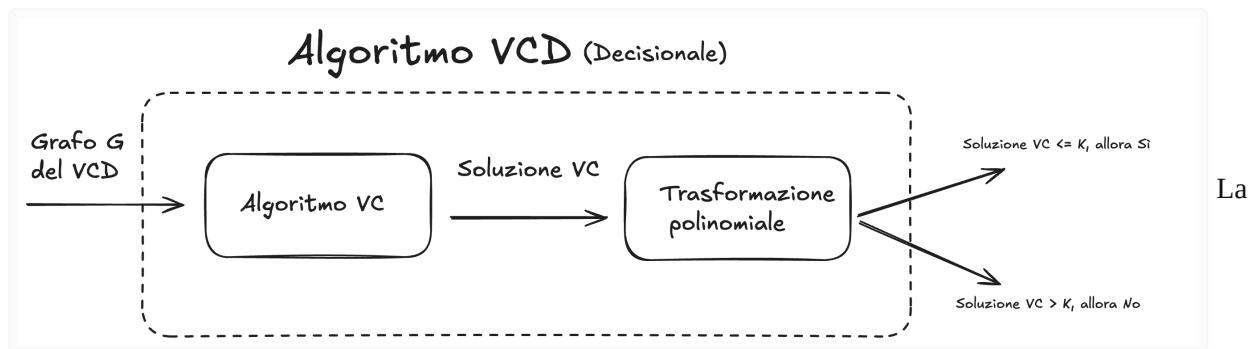
Nel nostro corso classifichiamo il problema decisionale del Vertex Cover come **NP-Completo**. Cosa significa per il tuo esame?

1. È in **NP**: Se ti do una soluzione (un insieme di vertici), puoi verificare in tempo polinomiale se copre tutti gli archi.
2. È **NP-Hard**: È difficile quanto il problema più difficile in NP (spesso si dimostra riducendo il problema della *Clique* o *3-SAT* al Vertex Cover).

In pratica: non conosciamo (e probabilmente non esiste) un algoritmo che trovi la soluzione *esatta* (il minimo assoluto) in tempo polinomiale. Se il grafo è grande, trovare l'ottimo richiederebbe tempi astronomici.

Teorema: Il problema del Vertex Cover (ottimizzazione) è NP-Hard.

Dato che la versione decisionale del Vertex Cover è NP-completo, definiamo la riduzione $VCD \leq_p VC$. Il Vertex Cover (ottimizzazione) è difficile almeno quanto il Vertex Cover Decisionale.



riduzione è molto semplice perché l'istanza dell'input dei due algoritmi è la stessa, ci serve solo definire una trasformazione polinomiale dell'output.

Quindi: 1. Utilizziamo l'algoritmo VC sul grado G per trovare il Vertex Cover $V' \subseteq V$ di cardinalità minima 2. Definiamo la trasformazione polinomiale dell'output: - se $|V'| \leq k$ allora rispondo **sì**. - se $|V'| > k$ allora rispondo **no**.

Una volta trovato il Vertex Cover con cardinalità minima, possiamo confrontarlo con k e quindi fornire la risposta decisionale del Vertex Cover. Il numero di chiamate dell'algoritmo dell'algoritmo è costante (una volta sola viene chiamato) e anche il numero di operazioni extra è costante. Dunque possiamo definire anche l'algoritmo del Vertex Cover np-hard.

Approfondimento

Possiamo definire anche dimostrare il contrario: $VC \leq_p VCD$ Anche la versione decisionale del Vertex Cover è difficile almeno quanto la versione di ottimizzazione.

Self-Reduction di Vertex Cover (Approfondimento)

Dimostriamo che il problema del Vertex Cover di ottimizzazione è egualmente difficile come il Vertex Cover decisionale.

Il problema di ricerca del Vertex Cover (VC_{search}) è polinomialmente riducibile al problema decisionale del Vertex Cover (VC_{decision}).

$$VC_{\text{search}} \leq_P VC_{\text{decision}}$$

Ciò implica che se esiste un algoritmo polinomiale per decidere se esiste una copertura di dimensione k , esiste anche un algoritmo polinomiale per trovare i vertici che compongono tale copertura.

Ipotesi: Sia $A(G, k)$ un algoritmo "oracolo" per VC_{decision} che accetta in input un grafo G e un intero k , e restituisce: - **YES**: se esiste un Vertex Cover di dimensione $\leq k$. - **NO**: se non esiste.

Procedimento

L'algoritmo di ricerca si svolge in due fasi sequenziali.

Fase 1: Calcolo della Cardinalità Minima (k_{opt})

L'obiettivo è determinare la dimensione minima esatta del Vertex Cover.

Poiché la dimensione della soluzione è un intero compreso tra 0 e $|V|$, utilizziamo una Ricerca Binaria interrogando l'oracolo A .

1. Inizializziamo l'intervallo di ricerca: $min = 0, max = |V|$.
2. Mentre $min < max$:
 - Poniamo $mid = \lfloor \frac{min+max}{2} \rfloor$.
 - Eseguiamo $A(G, mid)$.
 - Se l'esito è **YES**: la soluzione ottima è $\leq mid$, quindi poniamo $max = mid$.
 - Se l'esito è **NO**: la soluzione ottima è $> mid$, quindi poniamo $min = mid + 1$.
3. Al termine, $k_{opt} = min$.

Costo Fase 1: $O(\log |V|) \times Costo(A)$.

Fase 2: Costruzione dell'Insieme di Copertura (C)

L'obiettivo è individuare quali specifici vertici compongono l'insieme C di cardinalità k_{opt} .

Iteriamo su ogni vertice $v \in V$ del grafo originale $G = (V, E)$.

1. Inizializziamo $C = \emptyset$ e $k = k_{opt}$.
2. Per ogni vertice $v \in V$:
 - Costruiamo temporaneamente il grafo $G' = G \setminus v$ (rimuovendo v e i suoi archi incidenti).
 - Interrogiamo l'oracolo: $A(G', k - 1)$.
 - **Analisi della risposta:**
 - **Esito YES:** Significa che è possibile coprire i restanti archi con i $k - 1$ vertici rimanenti. Dunque, v può far parte di una soluzione ottima.
 - **Azione:** Aggiungiamo v a C ($C \leftarrow C \cup v$).
 - Decrementiamo il budget ($k \leftarrow k - 1$).
 - Aggiorniamo il grafo corrente rimuovendo definitivamente v ($G \leftarrow G'$).
 - **Esito NO:** Significa che rimuovendo v , il resto del grafo richiede ancora k vertici per essere coperto. Non abbiamo "guadagnato" nulla spendendo un vertice su v .
 - **Azione:** v non viene incluso in C . Il grafo G e il budget k rimangono invariati per la prossima iterazione.
3. Restituiamo C .

Costo Fase 2: $O(|V|) \times Costo(A)$.

Conclusione sulla Complessità

L'intero procedimento richiede un numero di chiamate all'oracolo pari a $O(\log |V| + |V|)$, che è polinomiale rispetto alla dimensione dell'input. Pertanto, la riduzione è polinomiale.

Approssimazione di Vertex Cover

Ci sono vari modi in cui possiamo andare a definire un algoritmo di approssimazione: - Algoritmi greedy - Programmazione Lineare intera

Algoritmi greedy

La strategia **Greedy** (ingorda) è spesso la prima tecnica utilizzata per tentare di approssimare problemi di ottimizzazione: si costruisce la soluzione passo dopo passo, effettuando ad ogni iterazione la scelta che appare localmente migliore. Tuttavia, per il problema del Vertex Cover, la definizione di "scelta migliore" è determinante. Analizziamo due euristiche naturali e dimostriamo perché **non garantiscono** un fattore di approssimazione costante.

La strategia consiste nel *scegliere un determinato nodo* del grafo con una **certa strategia**, poi rimuovere i vari archi collegati. Questo viene ripetuto fino a quando vengono rimossi tutti gli archi del grafo.

Approccio naive

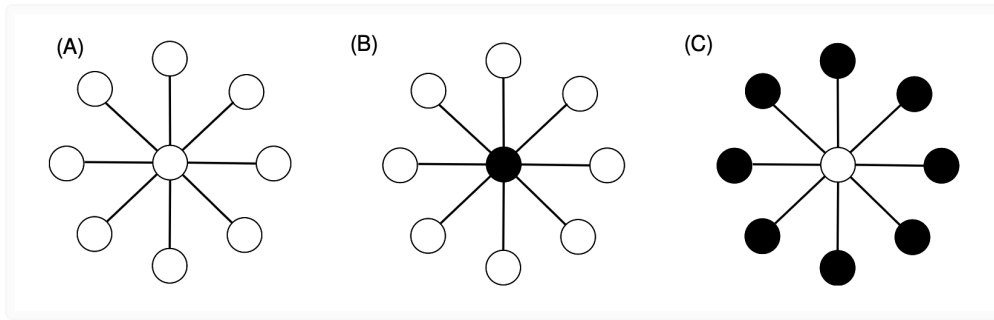
Scelgo un nodo qualsiasi con un grado maggiore di 0

```
Algorithm Greedy-VC-Naive( $G = (V, E)$ )
   $C = \{\}$ 
  WHILE ( $E$  is not empty) DO
    1. Scegli un vertice arbitrario  $v$  in  $V$  che abbia grado  $> 0$ 
    2.  $C = C \cup \{v\}$ 
    3. Rimuovi da  $E$  tutti gli archi incidenti a  $v$ 
  RETURN  $C$ 
```

Il costo computazionale dell'algoritmo è $O(|V| + |E|)$.

Sebbene l'algoritmo produca sempre una copertura valida in tempo polinomiale, il fattore di approssimazione non è limitato (non è costante).

Controesempio (Grafo a Stella):



Consideriamo un grafo a stella S_k composto da un nodo centrale c collegato a k foglie $l_1, l_2, \dots, l_k$.

- **Soluzione Ottima (C^*):** Basta selezionare il solo nodo centrale c . Dimensione $|C^*| = 1$.
- **Caso Peggior dell'Algoritmo:** L'algoritmo potrebbe sfortunatamente selezionare, uno dopo l'altro, tutti i nodi foglia $l_1, \dots, l_k$. Ogni volta che seleziona una foglia, copre solo un arco.
- **Risultato:** L'algoritmo restituisce un insieme di dimensione k .

Rapporto di Approssimazione:

$$\rho = \frac{|C_{greedy}|}{|C^*|} = \frac{k}{1} = k$$

Poiché k dipende dal numero di vertici del grafo (n), il rapporto di approssimazione è $O(n)$, il che è **inaccettabile** per un algoritmo di approssimazione.

Approccio euristico

Per ovviare al problema della stella, è naturale raffinare la scelta greedy: invece di un nodo a caso, scegliamo il nodo che "paga di più", ovvero quello che copre il maggior numero di archi in un colpo solo.

```

Algorithm Greedy-VC-MaxDegree( $G = (V, E)$ )
   $C = \{\}$ 
  WHILE ( $E$  is not empty) DO
    1. Scegli il vertice  $v$  in  $V$  con il grado corrente massimo
    2.  $C = C \cup \{v\}$ 
    3. Rimuovi da  $E$  tutti gli archi incidenti a  $v$ 
    4. Aggiorna i gradi dei vertici rimanenti
  RETURN  $C$ 

```

Questa strategia risolve brillantemente il caso della stella (selezionerebbe subito il centro). Tuttavia, è stato dimostrato che anche questa euristica **non ottiene un fattore di approssimazione costante**.

Il Limite Teorico (Perché fallisce):

Mentre per la stella funziona, esistono grafi appositamente costruiti in cui l'algoritmo viene "ingannato" nel preferire una lunga sequenza di nodi a grado medio-alto, invece di pochi nodi ottimi.

È noto in letteratura (dimostrazione di Chvátal, 1979 per Set Cover, applicabile qui) che questo algoritmo ha un fattore di approssimazione logaritmico:

$$\rho \approx \ln(n)$$

Dove $\ln(n)$ è il logaritmo naturale del numero di vertici. Sebbene $\ln(n)$ cresca molto lentamente, non è un numero costante (come 2 o 1.5). Per grafi molto grandi, l'errore può diventare arbitrariamente grande.

Il "Grafo Ingannevole" (Come l'algoritmo viene fregato) (Super approfondimento)

Il Vertex Cover è un caso particolare del Set Cover (dove gli "elementi da coprire" sono gli archi e gli "insiemi" sono i nodi). Poiché l'approccio greedy per il Set Cover ha un'approssimazione pari a H_d (l' n -esimo numero armonico, dove d è il grado massimo), il fattore di approssimazione è effettivamente $O(\log n)$.

Per visualizzare il limite teorico, è utile capire come viene costruito il grafo che "inganna" l'algoritmo.

Immagina un grafo bipartito con due insiemi di nodi, **A** e **B**:

1. **L'insieme A** contiene un numero piccolo di nodi, diciamo k . (Questo sarà il Vertex Cover ottimo).
2. **L'insieme B** contiene molti più nodi, suddivisi in "livelli" basati sul loro grado.
3. Gli archi sono collegati in modo tale che, all'inizio, i nodi in **B** abbiano un grado leggermente superiore ai nodi in **A**.

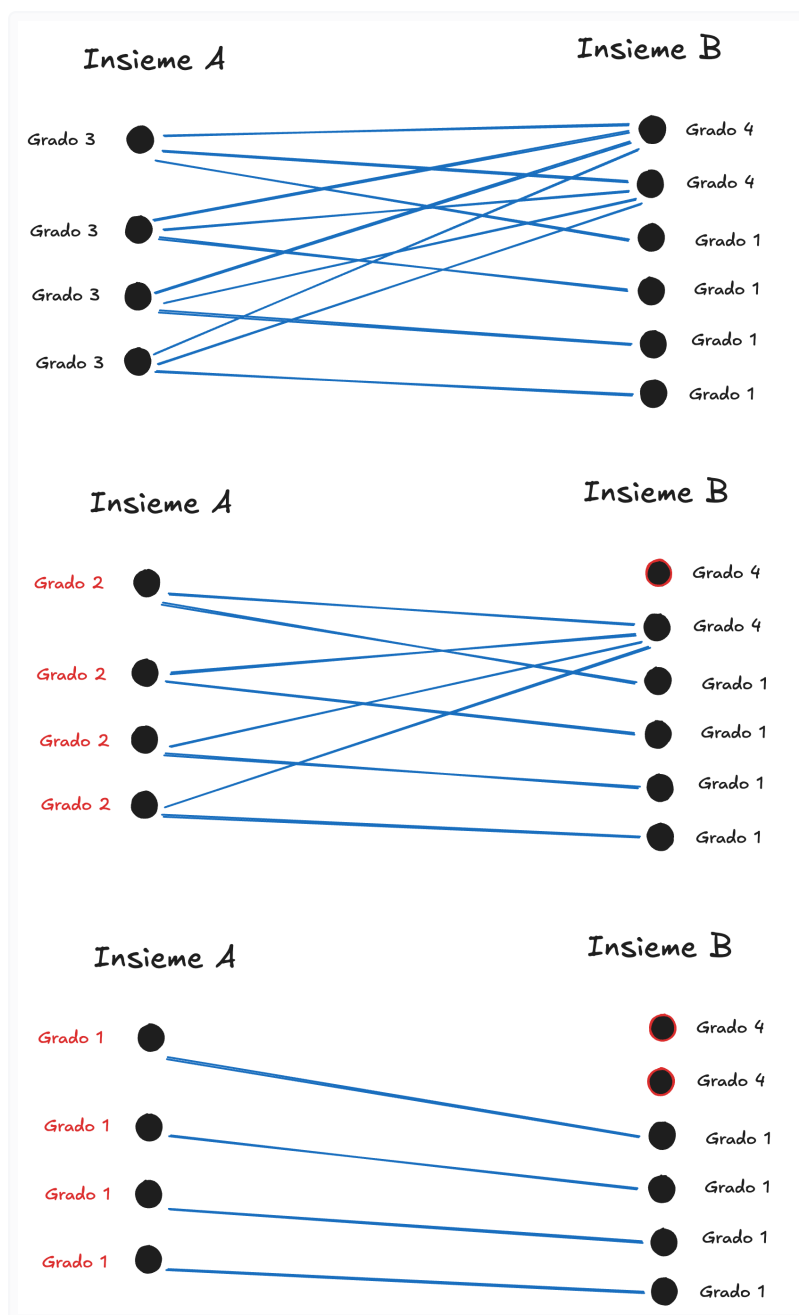
Cosa fa l'algoritmo?

L'algoritmo Greedy guarderà il grafo e dirà: *"I nodi in B hanno grado più alto, ne prendo uno da B"*.

Una volta rimosso quel nodo e i suoi archi, i gradi si aggiornano, ma la struttura è creata ad arte affinché ci sia *sempre* un nodo in **B** che sembra migliore dei nodi in **A**, passo dopo passo.

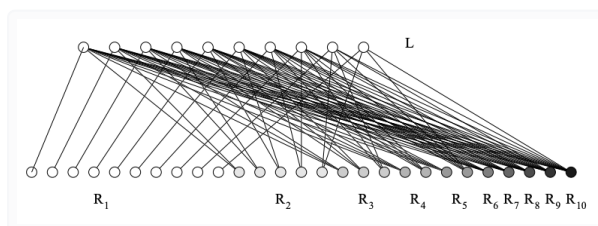
Alla fine, l'algoritmo sceglie quasi tutti i nodi dell'insieme B (che sono tantissimi), mentre la soluzione ottima era prendere semplicemente i pochi nodi dell'insieme A. È così che si accumula il fattore logaritmico $\ln(n)$.

Quello che succede in scala molto piccola è più o meno questo:



Il vero grafo bipartito che inganna l'algorithm dovrebbe essere questo:

Caption



Preso da

<https://cgi.csc.liv.ac.uk/~michele/TEACHING/COMP309/2005/Lec10.4.4.pdf>

Approccio Euristico corretto: Usiamo gli archi!

L'idea è semplice ma potente: troviamo un insieme di archi disgiunti (che non condividono vertici) e prendiamo entrambi gli estremi di questi archi.

Ecco lo pseudocodice formale:

```
APPROX-VERTEX-COVER(G)
1.  C ← ∅
2.  E' ← E[G]
3.  while E' ≠ ∅ do
4.    Scegli un arco arbitrario (u, v) ∈ E'
5.    C ← C ∪ {u, v}
6.    Rimuovi da E' ogni arco incidente su u o su v
7.  return C
```

Analisi della Complessità:

Il tempo di esecuzione è $O(V + E)$. Possiamo rappresentare il grafo con liste di adiacenza e marcare i vertici man mano che vengono rimossi/coperti. Visitiamo ogni vertice e ogni arco un numero costante di volte.

Correttezza e Analisi del Fattore di Approssimazione

Teorema: `APPROX-VERTEX-COVER` è un algoritmo di approssimazione polinomiale con fattore $\rho = 2$.

Dimostrazione:

Sia C l'insieme dei **vertici** restituiti dall'algoritmo. Sia C^* (o OPT) una copertura dei **vertici ottima** (di cardinalità minima). Sia A l'insieme degli **archi** selezionati nella riga 4 dell'algoritmo (questo insieme A costituisce un *Matching Massimale*).

Passo 1: Relazione tra C e A

L'algoritmo inserisce in C esattamente i due estremi di ogni arco scelto in A . Poiché nessun arco viene scelto due volte e gli archi in A sono disgiunti:

$$|C| = 2 \cdot |A| \quad (\text{Eq. 1})$$

Passo 2: Relazione tra C^* e A (Lower Bound)

Qualsiasi Vertex Cover valido deve coprire tutti gli archi di G , inclusi quelli in A .

Poiché gli archi in A sono disgiunti (non condividono vertici), nessun singolo vertice può coprire più di un arco di A .

Di conseguenza, per coprire $|A|$ archi disgiunti, sono necessari almeno $|A|$ vertici distinti.

Quindi, la soluzione ottima deve avere cardinalità:

$$|C^*| \geq |A| \quad (\text{Eq. 2})$$

(Nota: Questo è il passaggio che corregge il refuso delle slide della prof).

Passo 3: Conclusione

Combinando l'Eq. 1 e l'Eq. 2:

Dalla (Eq. 2) sappiamo che $|A| \leq |C^*|$. Sostituiamo $|A|$ nella (Eq. 1):

$$|C| = 2 \cdot |A| \leq 2 \cdot |C^*|$$

Riscrivendo in termini di rapporto di approssimazione:

$$\frac{|C|}{|C^*|} \leq 2$$

C.V.D. (Come Volevasi Dimostrare).

L'Analisi è "Tight" (Stretta)?

Un professore potrebbe chiederti: *"Possiamo fare meglio di 2 con questo specifico algoritmo?"* o *"L'analisi è pessimistica?"*

La risposta è: **Sì, l'analisi è tight**. Ci sono casi in cui l'algoritmo sbaglia esattamente di un fattore 2.

Esempio Formale (Il Grafo Bipartito Completo $K_{n,n}$):

Immagina un grafo con due insiemi di vertici $U = u_1, \dots, u_n$ e $W = w_1, \dots, w_n$, dove tutti gli u sono collegati a tutti i w .

- **Soluzione Ottima (OPT):** Basta prendere tutti i vertici di U (o tutti quelli di W). Cardinalità = n .
- **Soluzione Algoritmo Greedy:** L'algoritmo potrebbe sfortunatamente scegliere una serie di archi disgiunti come $(u_1, w_1), (u_2, w_2), \dots, (u_n, w_n)$. Per ogni arco, prende *entrambi* i vertici. Cardinalità = $2n$.

Rapporto: $2n/n = 2$.

Questo dimostra che non possiamo dimostrare un fattore inferiore a 2 per questo algoritmo specifico.

Vertex Cover Non Pesato

Non approfondito e non è materiale d'esame

Approssimazione Vertex Cover: "Relax & Round"

Il problema del Vertex Cover (copertura dei vertici) ci chiede di trovare il numero minimo di vertici per "toccare" tutti gli archi del grafo. Sappiamo che trovare la soluzione ottima esatta è **NP-Hard**. Non possiamo farlo velocemente. Gli altri metodi di approssimazione hanno un fattore di approssimazione troppo alto, quindi proviamo a cambiare strategia.

L'idea geniale è questa:

1. **Formuliamo** il problema come se fosse una scelta binaria rigida (prendo o non prendo il nodo). Questo è difficile (**Programmazione Lineare Intera - ILP**).

2. **Rilassiamo** (Relax) le regole: permettiamo di prendere "frazioni" di nodi (es. prendo mezzo nodo). Questo diventa facile da risolvere (**Programmazione Lineare - LP**).
3. **Arrotondiamo** (Round): convertiamo le frazioni in scelte binarie (0 o 1) per ottenere una soluzione valida reale (ammissibile al problema ILP).

Approssimiamo l'algoritmo

Vediamo i dettagli formali per applicare l'approssimazione Relax&Round per il VertexCover

Passo 1: Formulazione ILP (Integer Linear Programming)

Immagina di associare a ogni vertice v una variabile x_v .

- $x_v = 1$: il vertice è nel cover.
- $x_v = 0$: il vertice non è nel cover.

Vogliamo minimizzare il numero di vertici scelti ($\sum x_v$) con il vincolo che ogni arco (u, v) sia coperto, cioè almeno uno dei due estremi deve essere scelto ($x_u + x_v \geq 1$).

Il vincolo difficile è che $x_v \in \{0, 1\}$. Questo vincolo di "interezza" rende il problema difficile.

Abbiamo quindi:

Formulazione ILP di Vertex Cover
$$\min \sum_{v \in V} x_v \text{ s.t. } x_v + x_u \geq 1 \quad \forall (v, u) \in E \quad x_v \in \{0, 1\} \quad \forall v \in V$$

Passo 2: Rilassamento (Relax)

Rimuoviamo il vincolo rigido $x_v \in \{0, 1\}$ e lo sostituiamo con $0 \leq x_v \leq 1$.

Abbiamo quindi:

Formulazione PL di Vertex Cover frazionario
$$\min \sum_{v \in V} x_v \text{ s.t. } x_v + x_u \geq 1 \quad \forall (v, u) \in E \quad x_v \in [0, 1] \quad \forall v \in V$$

Ora stiamo risolvendo un problema di **Programmazione Lineare (LP)**.

- Possiamo risolverlo in **tempo polinomiale** (è facile per un computer).
- Otterremo una soluzione ottima *frazionaria* x^* . Ad esempio, potremmo avere $x^*_A = 0.5$, $x^*_B = 0.5$, $x^*_C = 0$.

Nota importante: Il costo di questa soluzione ottima frazionaria ($Cost(LP)$) sarà sicuramente *minore o uguale* al costo della soluzione ottima intera ($Cost(OPT)$), perché abbiamo meno restrizioni.

Passo 3: Arrotondamento (Round)

Ora dobbiamo tornare alla realtà. Non possiamo prendere "0.5 vertici". Dobbiamo decidere: 0 o 1?

La regola è semplice:

- Se $x_v^* \geq 0.5$, allora impostiamo $x_v = 1$ (prendiamo il vertice).

- Se $x_v^* < 0.5$, allora impostiamo $x_v = 0$ (scartiamo il vertice).

Perché funziona (Correttezza/Ammissibilità)?

Dobbiamo garantire che tutti gli archi siano coperti.

Per ogni arco (u, v) , il vincolo LP diceva che $x_u + x_v \geq 1$.

Matematicamente, è impossibile che due numeri siano entrambi strettamente minori di 0.5 e la loro somma sia ≥ 1 .

Quindi, almeno uno tra x_u e x_v deve essere ≥ 0.5 .

Di conseguenza, il nostro algoritmo arrotonderà almeno uno dei due a 1, coprendo l'arco.

Ovviamente non siamo certi che la soluzione sia ottima, ma possiamo dire che sia ammissibile.

3. Analisi del Fattore di Approssimazione (Cruciale!)

Vogliamo dimostrare che il costo della soluzione restituita dal nostro algoritmo $Cost(APPROX)$ è al massimo il doppio del costo della soluzione ottima ideale $Cost(OPT)$.

Definiamo i tre "attori" in gioco:

1. $Cost(OPT)$: Il costo della soluzione ottima **Intera** (il "vero" ottimo del problema, spesso impossibile da calcolare).
2. $Cost(X^*)$: Il costo della soluzione ottima **Rilassata/Frazionaria** (calcolata tramite LP).
 - *Nota Bene*: La soluzione X^* è matematicamente perfetta per il problema rilassato, ma è quasi sempre **INAMMISSIBILE*** per il problema originale (non puoi comprare "mezzo nodo"). Tuttavia, ci serve come punto di riferimento matematico (Lower Bound).
3. $Cost(APPROX)$: Il costo della nostra soluzione **Arrotondata** (la soluzione reale che l'algoritmo restituisce, valida e intera).

Passo 1: Relazione tra Rilassato (LP) e Ottimo (ILP)

Il problema rilassato (LP) cerca il minimo in uno spazio di soluzioni molto più ampio di quello intero (ILP), poiché ammette anche le frazioni.

Essendo meno vincolato, il problema LP riesce a trovare un costo *più basso o uguale* rispetto a quando siamo costretti a usare solo numeri interi.

Quindi, il costo frazionario è un **Lower Bound** (limite inferiore) del costo vero:

$$Cost(X^*) \leq Cost(OPT)$$

(Intuizione: X^ costa poco perché usa frazioni; OPT costa di più perché deve rispettare le regole dell'interezza).*

Passo 2: Relazione tra Arrotondamento e Rilassato

Analizziamo il **costo introdotto dalla fase di arrotondamento** per ogni singolo vertice v .

La nostra regola è: se $x_v^* \geq 0.5 \implies x_v = 1$, altrimenti 0.

Confrontiamo il valore arrotondato x_v con il valore frazionario originale x_v^* :

- Se $x_v = 0$: il costo è nullo, la disuguaglianza $0 \leq 2 \cdot x_v^*$ regge sempre.
- Se $x_v = 1$: questo accade solo se $x_v^* \geq 0.5$.
 - Matematicamente: $1 \leq 2 \cdot 0.5$.
 - Quindi: $x_v \leq 2 \cdot x_v^*$.

In entrambi i casi, paghiamo al massimo il doppio del valore frazionario:

$$x_v \leq 2 \cdot x_v^* \quad \forall v \in V$$

Passo 3: Conclusione (La Catena di Disuguaglianze)

Sommiamo i costi su tutti i vertici per ottenere il costo totale della nostra soluzione ($Cost(APPROX)$):

$$\begin{aligned} Cost(APPROX) &= \sum_{v \in V} x_v \leq \sum_{v \in V} 2 \cdot x_v^* && \text{(per il} \\ \text{Passo 2)} &= 2 \cdot \sum_{v \in V} x_v^* = 2 \cdot Cost(X^*) \end{aligned}$$

Ora sostituiamo $Cost(X^*)$ usando la disuguaglianza del Passo 1*:

$$Cost(APPROX) \leq 2 \cdot Cost(X^*) \leq 2 \cdot Cost(OPT)$$

Teorema Dimostrato: L'algoritmo Relax & Round restituisce una soluzione valida con un costo che non supera mai il doppio dell'ottimo.

Bonus: spiegazione del fattore 2

Il fattore 2 non è casuale, ma è conseguenza diretta della necessità di garantire una soluzione valida (**Safety**).

1. Il Vincolo (Safety):

Per coprire un arco (u, v) , la somma delle variabili frazionarie deve essere ≥ 1 . Nel caso peggiore, la soluzione LP ripartisce il peso equamente: $x_u = 0.5, x_v = 0.5$.

Per non lasciare l'arco scoperto, siamo **costretti** a porre la soglia di arrotondamento a **0.5** (se fosse più alta, arrotonderemmo entrambi a 0 e l'arco non sarebbe coperto).

2. Il Calcolo (Efficiency):

Il fattore di approssimazione è il rapporto massimo tra il costo che paghiamo (1, il nodo intero) e il costo che aveva calcolato l'LP (la soglia minima che fa scattare l'acquisto).

$$\text{Fattore} = \frac{\text{Costo Arrotondato}}{\text{Soglia Minima}} = \frac{1}{0.5} = 2$$

In breve: Poiché dobbiamo accettare valori bassi fino a $1/2$ per garantire la correttezza, il nostro errore massimo sarà l'inverso di quel valore, ovvero 2.

BadNews. Non noto nessun algoritmo di approssimazione per Vertex Cover con fattore di approssimazione migliore di due, e non è noto se possibile approssimare meglio di due o no.

Riprendiamo il discorso sul commesso viaggiatore, poi al massimo lo spostiamo dopo

TSP generale è approssimabile?

TEOREMA Se $P \neq NP$, allora per ogni costante $\rho \geq 1$ non esiste un algoritmo di approssimazione polinomiale per il TSP generale con fattore ρ .

1. La riduzione (Costruzione del Grafo G')

Partiamo da un'istanza del problema del **Ciclo Hamiltoniano (HC)**: un grafo $G = (V, E)$. Vogliamo sapere se G ha un ciclo che tocca tutti i vertici. Costruiamo un nuovo grafo completo $G' = (V, E')$ per il TSP:

- Se un arco e era presente nel grafo originale G , gli assegniamo costo **1**.
- Se l'arco e **non** era in G , gli assegniamo un costo pesantissimo: $\rho|V| + 1$.

2. Analisi dei due casi

- **Caso SI (G ha un ciclo hamiltoniano)**: In G' esiste un tour di costo esattamente $|V|$ (composto da $|V|$ archi di costo 1). Questo è il valore ottimo OPT . Un algoritmo di approssimazione A con fattore ρ deve restituire una soluzione $SOL \leq \rho \cdot OPT$. Quindi $SOL \leq \rho \cdot |V|$.
- **Caso NO (G non ha un ciclo hamiltoniano)**: Qualsiasi tour in G' deve usare almeno un arco che non era in G (quelli che costano $\rho|V| + 1$). Il costo del tour ottimo in G' sarà almeno $(\rho|V| + 1) + (|V| - 1)$, che è **strettamente maggiore** di $\rho|V|$. Di conseguenza, anche l'algoritmo A dovrà restituire un valore $SOL > \rho|V|$.

3. Conclusione

Se avessimo l'algoritmo A , basterebbe guardare il costo della soluzione prodotta:

1. Se $\text{costo} \leq \rho|V|$, allora G ha un ciclo hamiltoniano.
2. Se $\text{costo} > \rho|V|$, allora G non lo ha.

Avremmo risolto un problema NP-completo in tempo polinomiale, il che implicherebbe $P = NP$. E questo **sarebbe assurdo**.

Contesto generale

Il **Travelling Salesman Problem (TSP)** è un classico problema di ottimizzazione NP-hard.

Abbiamo visto che, nel caso generale, **non è possibile progettare algoritmi di approssimazione con fattore costante**, a meno di risultati molto forti sulla complessità.

La buona notizia è che questo risultato negativo **non vale per tutte le istanze**. Esiste infatti un sottoinsieme di istanze particolarmente rilevanti, in cui la funzione costo sugli archi rispetta la **disuguaglianza triangolare**. In questo caso diventa possibile progettare algoritmi di approssimazione efficaci.

TSP con disuguaglianza triangolare (TSPdt o TSP metric)

Ricordiamo che si tratta del problema del tsp con una garanzia in più: la **disuguaglianza triangolare**.

Proprietà triangolare

La funzione costo sugli archi soddisfa:

$$\forall i, j, k \in V : \quad c(i, j) \leq c(i, k) + c(k, j)$$

Intuizione

Andare **direttamente** da i a j non costa più che passare per un nodo intermedio.

Definizione formale del problema

Problema del Commesso Viaggiatore con disuguaglianza triangolare (TSPdt)

Input - Grafo completo non diretto $G = (V, E)$ - Funzione costo $c : E \rightarrow \mathbb{R}^+$ - c soddisfa la disuguaglianza triangolare

Output - Un **ciclo hamiltoniano** su G di **costo minimo**

Approssimazione per TSPdt

Per questo problema, un algoritmo di approssimazione deve restituire **una soluzione ammissibile**, cioè un ciclo hamiltoniano, che può avere un costo maggiore di quello ottimo.

Nel seguito verranno presentati **due algoritmi progettati ad-hoc** per il TSPdt; il secondo rappresenta un miglioramento del primo.

Notazione sui costi

Dato un sottografo:

$$G' = (V', E') \subseteq G = (V, E)$$

si definisce il costo:

$$\text{cost}(G') = \sum_{e \in E'} c(e)$$

Concetto utile: Ciclo Euleriano

Un concetto fondamentale per entrambi gli algoritmi è quello di **ciclo euleriano**.

Un ciclo euleriano è un cammino chiuso che attraversa **esattamente una volta ciascun arco del grafo**.

Un risultato classico afferma che un grafo ammette un ciclo euleriano **se e solo se tutti i suoi nodi hanno grado pari**. Quando questa condizione è soddisfatta, il ciclo euleriano può essere trovato in **tempo polinomiale**.

Proprietà fondamentale della disuguaglianza triangolare

Teorema — Accorciamento dei cammini nei grafi metrici

Sia G un grafo con archi pesati tale che la funzione costo soddisfi la **disuguaglianza triangolare**, cioè:

$$\forall i, j, k \in V : \quad c(i, j) \leq c(i, k) + c(k, j)$$

Sia inoltre:

$$C = \langle v_1, v_2, \dots, v_{k+1} \rangle$$

un **cammino semplice** in G , composto da k archi.

Allora vale:

$$c(v_1, v_{k+1}) \leq \sum_{i=1}^k c(v_i, v_{i+1})$$

In altre parole, **il costo dell'arco diretto tra il primo e l'ultimo nodo del cammino non è maggiore del costo dell'intero cammino**.

Significato intuitivo

Il teorema formalizza un'idea molto naturale: nei grafi che rispettano la disuguaglianza triangolare, **fare deviazioni non conviene**.

Se si percorre un cammino passando per nodi intermedi, il costo complessivo non può essere inferiore a quello del collegamento diretto tra gli estremi.

Geometricamente, è l'equivalente del fatto che **la linea retta è il percorso più breve**.

Dimostrazione

Idea generale della dimostrazione

La dimostrazione si basa su un'**induzione sulla lunghezza del cammino**, cioè sul numero di archi k .

L'idea è quella di: - spezzare il cammino in parti più corte, - usare la disuguaglianza triangolare per eliminare nodi intermedi, - ridurre progressivamente il cammino fino a collegare direttamente i due estremi.

Caso base

Nel caso $k = 2$, il cammino è:

$$v_1 \rightarrow v_2 \rightarrow v_3$$

La tesi diventa:

$$c(v_1, v_3) \leq c(v_1, v_2) + c(v_2, v_3)$$

Questa è **esattamente la disuguaglianza triangolare**, quindi il caso base è immediatamente verificato.

Passo induttivo

Si assume che la tesi sia vera per tutti i cammini di lunghezza minore di k e si considera un cammino di lunghezza k :

$$C = \langle v_1, v_2, \dots, v_{k-1}, v_k, v_{k+1} \rangle$$

Il costo totale del cammino può essere scritto come:

$$\sum_{i=1}^k c(v_i, v_{i+1}) = \sum_{i=1}^{k-2} c(v_i, v_{i+1}) + c(v_{k-1}, v_k) + c(v_k, v_{k+1})$$

Applicando la disuguaglianza triangolare agli ultimi due archi si ottiene:

$$c(v_{k-1}, v_k) + c(v_k, v_{k+1}) \geq c(v_{k-1}, v_{k+1})$$

Inoltre, per ipotesi induttiva, il costo del sottocammino che va da v_1 a v_{k-1} è almeno:

$$c(v_1, v_{k-1})$$

Combinando questi risultati e applicando nuovamente la disuguaglianza triangolare si ottiene:

$$\sum_{i=1}^k c(v_i, v_{i+1}) \geq c(v_1, v_{k+1})$$

Questo conclude il passo induttivo.

Interpretazione Grafica

Ogni applicazione della disuguaglianza triangolare consente di **eliminare un nodo intermedio** dal cammino senza aumentare il costo.

Ripetendo questo processo, il cammino viene progressivamente “raddrizzato” fino a diventare un singolo arco tra i due estremi.

Importanza del teorema

Questo risultato è fondamentale negli algoritmi di approssimazione per il **TSP con disuguaglianza triangolare**, perché consente di: - visitare nodi più volte durante una costruzione intermedia (ad esempio con cicli euleriani), - eliminare successivamente le visite ridondanti, - ottenere un **ciclo hamiltoniano** senza aumentare il costo totale.

Il teorema giustifica formalmente la possibilità di “saltare” nodi già visitati mantenendo un buon controllo sul costo della soluzione.

Grazie alla disuguaglianza triangolare, ogni cammino può essere accorciato collegando direttamente i suoi estremi senza aumentare il costo totale.

Algoritmo di 2-approssimazione per TSP con disuguaglianza triangolare

Consideriamo il problema del **TSP con disuguaglianza triangolare** (TSPdt), cioè su un grafo completo $G = (V, E)$ con funzione di costo c tale che:

$$c(u, w) \leq c(u, v) + c(v, w) \quad \forall u, v, w \in V$$

Descrizione dell'algoritmo (MST-based)

L'algoritmo di 2-approssimazione procede come segue:

1. Si calcola un **Minimum Spanning Tree** T di G e si sceglie un nodo r come radice;
 2. Si **raddoppiano tutti gli archi di** T , ottenendo un multigrafo in cui ogni nodo ha grado pari;
 3. Si calcola un **ciclo euleriano** E sul multigrafo;
 4. A partire da E , si costruisce un **ciclo hamiltoniano** H saltando i nodi già visitati (shortcut);
 5. Si restituisce H .
-

Concetti da ripassare

Minimum Spanning Tree (MST)

Definizione Dato un grafo **non orientato**, **connesso** e pesato $G = (V, E)$ con pesi $w(e) \geq 0$, un **Minimum Spanning Tree** è un sottoinsieme di archi $T \subseteq E$ tale che: - T connette tutti i vertici (è uno **spanning tree**), - non contiene cicli, - ha costo totale minimo:

$$w(T) = \sum_{e \in T} w(e)$$

tra tutti gli spanning tree di G .

Uno spanning tree su $n = |V|$ nodi ha sempre **esattamente** $n - 1$ archi.

Intuizione L'MST è il modo **più economico** di collegare tutti i nodi senza “sprechi” (cioè senza cicli). Non è necessariamente unico: se ci sono pesi uguali, possono esistere più MST.

Proprietà chiave (utili per dimostrazioni)

Cut Property (proprietà del taglio) Considera un taglio $(S, V \setminus S)$.

Se e è l'arco di **peso minimo** che attraversa quel taglio, allora esiste un MST che contiene e (in molte formulazioni: e è “safe”).

Questa proprietà giustifica perché algoritmi greedy come Kruskal/Prim funzionano.

Cycle Property (proprietà del ciclo) In qualunque ciclo, un arco di **peso massimo** nel ciclo **non** può appartenere a un MST (se è strettamente maggiore degli altri), perché togliendolo resti connesso ma spendi meno.

Algoritmi classici

Kruskal (edge-based) Idea greedy: ordina gli archi per peso crescente e aggiungili se **non creano un ciclo** (tipicamente con Union-Find). - Output: una foresta che diventa albero quando raggiunge $n - 1$ archi. - Tempo tipico: $O(m \log m)$ (ordinamento), con $m = |E|$.

Prim (vertex-based) Idea greedy: cresci un albero partendo da una radice, aggiungendo ogni volta l'arco minimo che collega l'albero a un nodo esterno (con coda di priorità). - Tempo tipico: $O(m \log n)$ con heap.

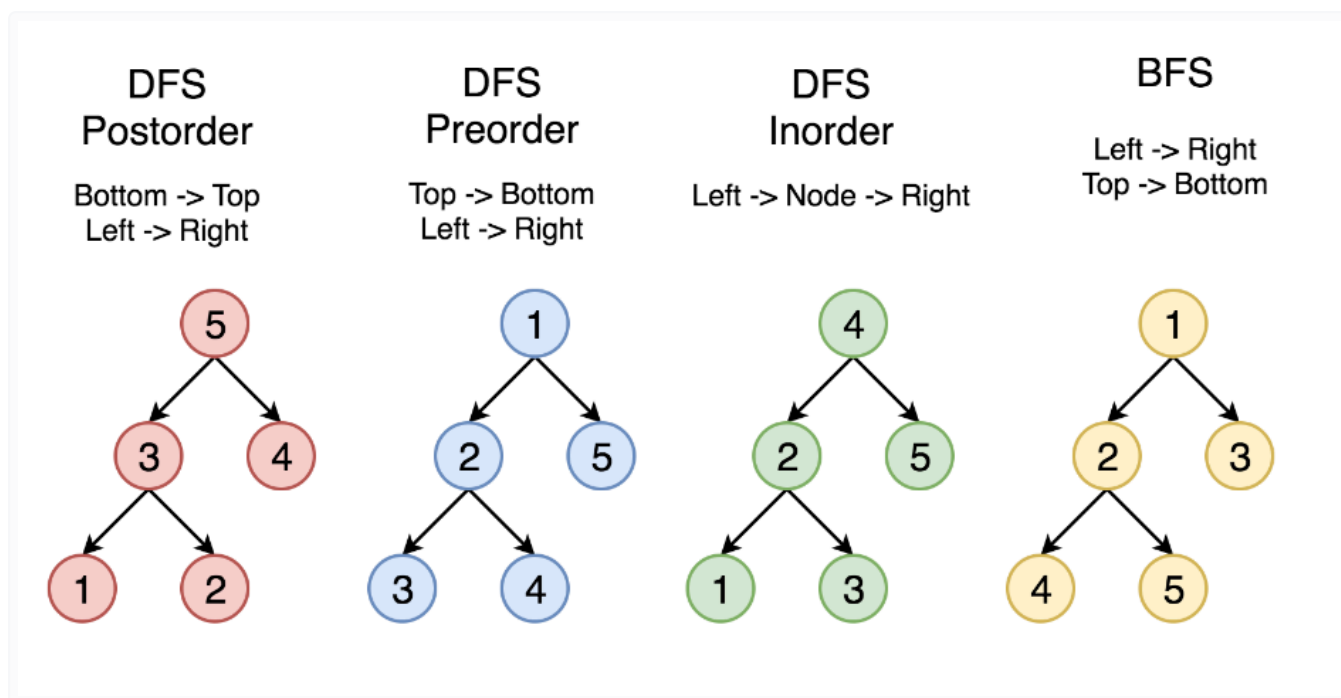
Perché serve nel TSP con disuguaglianza triangolare

Nel 2-approx per TSP si usa l'MST perché è un **lower bound** sul costo del tour ottimo $w(H^*)$:
: $w(T) \leq w(T') \leq w(H^*)$ dove T è un MST e T' è un tour.

poiché T è minimo,

$w(T) \leq w(T') \leq w(H^*)$ Quindi $w(T) \leq w(H^*)$.

DFS (Depth-First Search) e ordini di visita



Idea della DFS La **DFS** (*Depth-First Search*, ricerca in profondità) esplora un grafo/albero seguendo questa strategia: da un nodo corrente si visita **un vicino non ancora visitato** e si continua “andando avanti” il più possibile lungo quel ramo. Quando non ci sono più vicini non visitati, si **torna indietro** (backtracking) all’ultimo nodo che aveva ancora alternative e si riprende da lì.

Nel caso dei **grafi**, la DFS deve mantenere un insieme/array `visited` per evitare di entrare in cicli e rivisitare nodi già esplorati.

Ordini di visita: pre-order, post-order, in-order

- **Pre-order:** Nodo → Figli
- **Post-order:** Figli → Nodo
- **In-order (binario):** Sinistra → Nodo → Destra

Correttezza dei passi dell’algoritmo

(a) Esistenza e calcolo del ciclo euleriano

Dopo il raddoppio degli archi di T , ogni nodo ha grado pari, perché il grado di ciascun nodo raddoppia rispetto a quello nell’albero.

Per il **teorema di Eulero**, un grafo connesso in cui tutti i nodi hanno grado pari ammette un ciclo euleriano. Il ciclo E può essere calcolato in tempo polinomiale, ad esempio seguendo una visita DFS in pre-ordine dell’albero T .

(b) Costruzione del ciclo hamiltoniano

Il ciclo hamiltoniano H si ottiene visitando i nodi nell'ordine in cui compaiono in E , includendo ogni nodo **solo la prima volta** e saltando le visite successive.

Questa operazione è sempre possibile perché il grafo di partenza G è **completo**: tra ogni coppia di nodi esiste un arco, anche se non appartiene a E .

Grazie alla disuguaglianza triangolare, la sostituzione di un cammino con un arco diretto (**shortcut**) non aumenta il costo.

Teorema – Fattore di approssimazione

Teorema. L'algoritmo descritto è una **2-approssimazione** per il problema TSPdt.

Dimostrazione

La dimostrazione si basa su due osservazioni fondamentali.

(1) Bound superiore sul costo della soluzione approssimata

Il ciclo euleriano E percorre ogni arco di T esattamente due volte, quindi:

$$\text{cost}(E) = 2 \cdot \text{cost}(T)$$

.

Poiché il ciclo hamiltoniano H si ottiene da E tramite shortcut che non aumentano il costo, vale:

$$\text{cost}(H) \leq \text{cost}(E) \leq 2 \cdot \text{cost}(T)$$

.

(2) Bound inferiore sul costo della soluzione ottima

Sia H^* un ciclo hamiltoniano ottimo. Rimuovendo un arco da H^* otteniamo un albero di copertura T' , per cui $\text{cost}(T') \leq \text{cost}(H^*)$

Poiché T è un minimum spanning tree, vale anche:

$$\text{cost}(T) \leq \text{cost}(T')$$

Combinando le disuguaglianze otteniamo:

$$\text{cost}(H^*) \geq \text{cost}(T)$$

Conclusione

Dalle due osservazioni segue:

$$\text{cost}(H) \leq 2 \cdot \text{cost}(H^*)$$

Tightness dell'analisi

Il fattore di approssimazione 2 è **tight**, cioè non migliorabile per questo algoritmo.

Esempio

Consideriamo un grafo completo con n nodi e pesi definiti come segue: - $c(1, v) = 1$ per ogni $v = 2, \dots, n$; - $c(v, v+1) = 1$ per $v = 1, \dots, n-1$ e $c(n, 1) = 1$; - tutti gli altri archi hanno costo 2.

Esiste un ciclo hamiltoniano ottimo di costo:

$$\text{cost}(H^*) = n$$

Il minimum spanning tree può essere una **stella centrata nel nodo 1**, di costo $n-1$. Il ciclo euleriano risultante passa ripetutamente dal centro e, dopo le shortcut, il ciclo hamiltoniano approssimato utilizza: - $n-2$ archi di costo 2, - 2 archi di costo 1.

Il costo totale è quindi:

$$\text{cost}(H) = 2(n-2) + 2 = 2n-2$$

Il rapporto di approssimazione è:

$$\frac{\text{cost}(H)}{\text{cost}(H^*)} = \frac{2n-2}{n} \rightarrow 2 \quad \text{per } n \rightarrow \infty$$

Osservazione finale

Questo algoritmo è concettualmente semplice ma **non sfrutta informazioni strutturali più profonde** del problema. Per migliorare il fattore di approssimazione è necessario introdurre tecniche più raffinate, come nel caso dell'algoritmo di **Christofides (3/2-approssimazione)**.

3.1.2 Algoritmo di Christofides (TSPdt) — 3/2-approssimazione

Contesto e assunzioni

Consideriamo **TSP con disuguaglianza triangolare** su grafo completo $G = (V, E)$ con costi c tali che:
 $c(u, w) \leq c(u, v) + c(v, w) \quad \forall u, v, w \in V$. L'idea è costruire un **multigrafo euleriano più economico** rispetto al "raddoppia-MST" della 2-approssimazione.

Matching e Perfect Matching

- **Matching**: insieme di archi a due a due disgiunti (nessun vertice è incidente a più di un arco del matching).
 - **Perfect matching**: matching che **copre tutti i vertici** (ogni vertice è incidente esattamente a un arco del matching). Condizione necessaria: il numero di nodi deve essere **pari**.
-

Algoritmo di Christofides (schema)

1. Calcola un **MST** T^* di G .
 2. Sia V_d l'insieme dei nodi di **grado dispari** in T^* . Considera il sottografo completo G_d indotto da V_d .
 3. Calcola un **minimum-weight perfect matching** M^* su G_d .
 4. Costruisci il multigrafo $G' = T^* \cup M^*$ (unione multinsieme degli archi).
 5. Calcola un **ciclo euleriano** E in G' .
 6. Da E costruisci un **ciclo hamiltoniano** H facendo **shortcut** (salti i nodi già visitati).
 7. Restituisci H .
-

Perché l'algoritmo è sempre eseguibile

(a) Esistenza del perfect matching su G_d

Serve che $|V_d|$ sia pari. Questo vale sempre perché in qualunque grafo la somma dei gradi è $2|E|$ (pari).

Separando la somma dei gradi pari e la somma dei gradi dispari, la seconda deve essere pari; una somma di numeri dispari è pari **solo se** ci sono **un numero pari di termini**, quindi i nodi di grado dispari sono in numero pari.

Dato che G_d è completo sui vertici V_d , l'esistenza del perfect matching segue, e il matching minimo si calcola in tempo polinomiale con algoritmi noti.

(b) Esistenza del ciclo euleriano in G'

In T^* ogni nodo in V_d ha grado dispari. Aggiungendo M^* , ogni nodo di V_d riceve **esattamente un arco in più**, quindi diventa di grado pari; i nodi già pari restano pari. Quindi **tutti i gradi in G' sono pari** $\Rightarrow$ esiste un ciclo euleriano E .

(c) Shortcut per ottenere un ciclo hamiltoniano

Come nella 2-approssimazione: seguendo l'ordine di E , si include un nodo solo la prima volta. Essendo G completo e valendo la triangolare, gli shortcut sono sempre possibili e non aumentano il costo.

Teorema: fattore di approssimazione $3/2$

Teorema. Christofides è una $3/2$ -approssimazione per TSP.

Idee chiave della prova

Sia H^* un tour ottimo.

1) Lower bound da MST

Togliendo un arco da H^* otteniamo un *spanning tree*, quindi $\text{cost}(H^*) \geq \text{cost}(T^*)$.

- 2) **Bound sul matching minimo** Considera V_d (nodi dispari di $T^\wedge$). *Dal tour ottimo $H^\wedge$ otteniamo tramite shortcut un ciclo Γ che visita solo V_d : $\text{cost}(\Gamma) \leq \text{cost}(H^\wedge)$ (triangolare).*
- Poiché $|V_d|$ è pari, Γ ha lunghezza pari e si può decomporre alternando gli archi in due matching disgiunti M_1 e M_2 : $\text{cost}(\Gamma) = \text{cost}(M_1) + \text{cost}(M_2) \leq 2 \cdot \text{cost}(M^\wedge)$, da cui: $\text{cost}(M^\wedge) \leq \frac{\text{cost}(H^\wedge)}{2}$.
- 3) **Costo del tour euleriano e shortcut** Il ciclo euleriano usa tutti gli archi di $T^\wedge$: $\text{cost}(E) = \text{cost}(T^\wedge) + \text{cost}(M^\wedge)$, e lo shortcut non aumenta: $\text{cost}(H) \leq \text{cost}(E)$.

Conclusione

$$\text{cost}(H) \leq \text{cost}(T^\wedge) + \text{cost}(M^\wedge) \leq \text{cost}(H^\wedge) + \frac{\text{cost}(H^\wedge)}{2} = \frac{3}{2} \text{cost}(H^\wedge).$$

Complessità (alto livello)

- MST: tipicamente $O(m \log n)$ (su completo $m = \Theta(n^2)$).
 - Step dominante: **minimum-weight perfect matching** su $|V_d| \leq n$, tipicamente $O(n^3)$ (come ordine di grandezza). Quindi il costo è dominato dal matching.
-

Note utili per l'orale

- Il “trucco” rispetto alla 2-approx è sostituire il raddoppio dell'MST con l'aggiunta di un matching minimo solo sui nodi dispari: così rendi euleriano il grafo spendendo “circa metà OPT” invece di “+MST intero”.
- La parte delicata della prova è mostrare $\text{cost}(M^*) \leq \text{OPT}/2$ usando Γ e la decomposizione in due matching.

Introduzione agli algoritmi distribuiti

Che cos'è un sistema distribuito

Un **sistema distribuito** è una collezione finita di entità computazionali eterogenee che comunicano tramite **scambio di messaggi** per raggiungere un obiettivo comune. L'obiettivo è sempre quello di progettare **algoritmi distribuiti** che siano: - **corretti** (risolvono davvero il problema richiesto), - **efficienti** (hanno un costo "piccolo" rispetto alle risorse disponibili).

Esempi di ambienti distribuiti: web, reti di comunicazione, sensor networks, reti robotiche, mobile agents.

Perché usare sistemi distribuiti

Motivazioni tipiche: - **condivisione di risorse** (stampanti, file, pagine web, ...), - **tolleranza ai guasti** (il sistema resta disponibile anche se alcune parti sono temporaneamente "out of order"), - **scalabilità** (in dimensione, distribuzione geografica, amministrazione).

Distribuito vs parallelo (da non confondere all'esame)

- **Macchina sequenziale**: 1 CPU, 1 memoria, operazioni sequenziali.
- **Sistema parallelo**: più processori con (tipicamente) **memoria condivisa**, distanze piccole → i problemi principali sono sincronizzazione e mutua esclusione.
- **Sistema distribuito**: molti processori con **memorie separate**, distanze grandi → la comunicazione via messaggi è centrale e introduce nuovi problemi (ritardi, coordinamento, conoscenza parziale).

In un ambiente distribuito emergono tre proprietà chiave: 1. **Molteplicità**: molte entità. 2. **Autonomia**: memoria privata e clock locali (non necessariamente sincronizzati). 3. **Interazione**: cooperazione tramite messaggi per "fare di più" rispetto al singolo nodo.

Il modello

Il modello standard del corso è: - una **rete** rappresentata come un **grafo** $G = (V, E)$, - nodi V = entità (processi/nodi/siti/agenti...), - archi E = link di comunicazione, - comunicazione **message-passing** (scambio di messaggi su link).

Proprietà "di base" delle entità

Ogni entità: - ha un **input** (anche vuoto), - esegue lo **stesso codice** (protocollo), - deve produrre lo **stesso tipo di output** (tutte risolvono lo stesso problema).

Stato

Ogni entità x ha un registro $status(x)$ che indica lo stato corrente. Esiste un insieme finito S di stati possibili (es. idle, waiting, computing, ...) e in ogni istante $x \in S$.

Eventi

Il comportamento è **reattivo**: senza evento non c'è azione. Eventi tipici: - **tick del clock** (interno), - **arrivo di un messaggio** (interno), - **impulso spontaneo** dall'ambiente (esterno).

Azioni

Un'azione è una sequenza di attività consentite (computare, inviare messaggi, cambiare stato, leggere/scrivere memoria interna, set/reset del clock, oppure NIL). Le azioni sono: - **atomiche** (non interrompibili), - **terminanti** (finiscono in tempo finito).

Comportamento e simmetria

Il **behaviour** (comportamento) di un'entità è l'insieme delle regole “per ogni stato e per ogni evento, che azione eseguo?”.

$$Stato, Evento \implies Azione$$

Molto spesso si assume un sistema **simmetrico**: tutte le entità hanno lo stesso behaviour (stesso protocollo). Differenze di ruolo (es. initiator vs sleeping) si modellano tramite stati/variabili locali.

Comunicazione

La comunicazione può essere unidirezionale o bidirezionale. Il **vicinato** di un nodo x è l'insieme dei nodi con cui x può comunicare, indicato spesso come $N(x)$. In particolare: - $N_o(x) \implies$ Vicini a cui mandare messaggi (out-neighbours) - $N_i(x) \implies$ Vicini da cui ricevere messaggi (in-neighbours)

$$N(x) = N_o(x) \cup N_i(x)$$

Assiomi del modello (cose sempre vere nel modello base)

Il **modello generale** non parte da “mille assunzioni”, ma da **pochi assiomi minimi**. Tutto ciò che aggiungi oltre questi assiomi si chiama **restrizione** (o *submodel*): rende il protocollo applicabile solo a certi sistemi/reti.

Assioma 1 — Ritardi di comunicazione finiti (Finite Communication/Transmission Delays)

Enunciato (idea): *in assenza di guasti*, un messaggio inviato da un'entità x a un suo out-neighbor y **arriva** a y in **tempo finito** e senza corruzione.

Cosa implica davvero - Non esiste “perdita” di messaggi **se** non ci sono failure. - Non puoi avere ritardi infiniti: prima o poi il messaggio viene ricevuto.

Cosa NON implica - Non dice che esista un limite superiore noto o costante: il ritardo può essere *arbitrariamente grande* e imprevedibile (quindi il sistema è naturalmente **asincrono**).

Conseguenza: se vuoi ragionare “a round” o con tempi deterministici, devi **aggiungere restrizioni temporali** (es. *Bounded delay Δ , unitary delay, synchronized clocks*).

Assioma 2 — Orientamento locale (Local Orientation)

Enunciato (idea): ogni entità riesce a **distinguere tra i propri vicini**: - distingue tra i suoi **out-neighbors** (a chi può inviare), - distingue tra i suoi **in-neighbors** (da chi può ricevere), e quando riceve un messaggio sa **da quale “porta”/vicino** è arrivato.

Interpretazione operativa (porte / port numbers) - Ogni nodo assegna etichette locali (porte) ai suoi link incidenti: sono **locali** e iniettive (porte diverse per vicini diversi). - Quindi puoi dire: “manda al vicino sulla porta 3” oppure “ho ricevuto dalla porta 2”.

Perché è fondamentale - Senza local orientation, molte primitive (flooding controllato, costruzione di alberi, instradamento su base locale) non sono nemmeno formulabili in modo utile: non potresti scegliere un vicino specifico, né distinguere chi ti ha scritto.

Assiomi vs restrizioni (da ricordare)

- **Assiomi**: proprietà minime del modello base (valgono sempre).
 - **Restrizioni**: proprietà aggiuntive usate dal protocollo (es. “link bidirezionali”, “total reliability”, “grafo connesso”, “IDs distinti”, “delay bounded Δ ...”). Ogni restrizione aumenta la “forza” del sistema ma riduce i casi in cui il protocollo è applicabile.
-

Restrizioni (assunzioni del modello): sempre esplicitarle

Nel distribuito, la (im)possibilità e la complessità dei problemi dipendono in modo cruciale dalle **restrizioni** assunte. Ogni protocollo va letto insieme alle sue assunzioni.

Categorie tipiche: - **Restrizioni di comunicazione**: es. ordinamento FIFO su un link; link bidirezionali (grafo non orientato). - **Restrizioni di affidabilità**: consegna garantita; modelli di affidabilità parziale/totale; meccanismi di fault detection (se presenti). - **Restrizioni topologiche / di conoscenza**: connettività del grafo; conoscenza di n , m , diametro, ecc. - **Restrizioni temporali**: asincrono (default del corso se non detto), delay bounded (esiste Δ), unitary delay, clock sincronizzati.

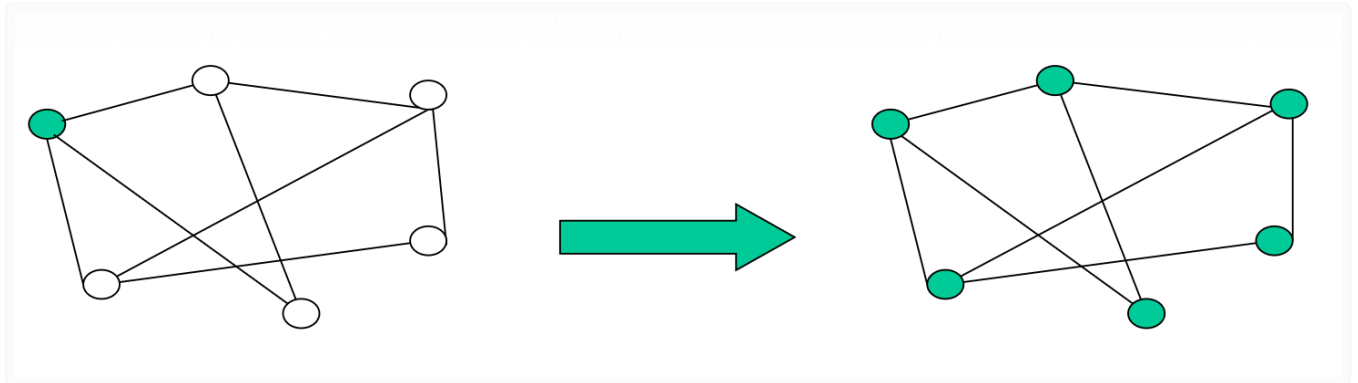
Misure di complessità nel distribuito

Per algoritmi sequenziali guardiamo tipicamente tempo e spazio; nel distribuito invece sono centrali: 1. **Amount of communication**: numero di messaggi scambiati (o più fine: numero di bit). 2. **Tempo**: ritardo massimo che possiamo avere durante la comunicazione. - *tempo ideale (sincrono)*: 1 unità di tempo per trasmettere 1 messaggio, - *tempo realistico (asincrono)*: i ritardi sono imprevedibili → spesso si conta la lunghezza della catena di messaggi* più lunga (dipendenza causale) in un’esecuzione.

N.B. A differenza dello studio di complessità degli algoritmi classici, mi concentro di più sulla comunicazione tra le varie entità, rispetto a considerare il costo computazionale nelle singole macchine.

Broadcast (Bcast) e protocollo FLOODING

Problema (Broadcast)



Dato un sistema distribuito su un grafo di comunicazione $G = (V, E)$, **un solo nodo** (l'**initiator**) possiede un'informazione I e vuole far sì che **tutti** gli altri nodi la apprendano **in tempo finito**.

Il requisito “un solo nodo inizia” è l'assunzione **Unique Initiator** (spesso indicata come UI^+ : l'initiator è proprio quello che inizialmente ha I).

Restrizioni tipiche (standard)

Per discutere Bcast e FLOODING si assumono spesso le **standard restrictions**

$$R = BL, CN, TR$$

- dove:
- **BL (Bidirectional Links)**: il grafo non è orientato
 - **CN (Connectivity)**: il grafo è connesso (tutti raggiungibili)
 - **TR (Total Reliability)**: nessun guasto durante l'esecuzione

Prima idea (ingenua)

Idea: INITIATOR invia I a tutti i vicini; ogni nodo SLEEPING che riceve I lo invia a tutti i vicini.

- Stati $S = \{\text{INITIATOR, SLEEPING}\}$
- Informazione I

Entità INITIATOR

```
quando parte:
    manda I a tutti i vicini
    diventa DONE

se riceve I:
    ignora
```

Entità SLEEPING

```
quando parte:
    dorme

se riceve I:
    manda I a tutti i vicini
```

Questa idea può **non terminare**: alcuni nodi possono continuare a rimandarsi *I* all'infinito (loop di messaggi).

Cosa vuol dire “algoritmo corretto” nel distribuito

Per la correttezza servono **due** proprietà: 1. **Safety / Solves the problem**: tutti apprendono *I* 2. **Termination**: l'esecuzione termina (almeno localmente)

Seconda idea: invio “una sola volta” + stato DONE

Per evitare loop: *quando un nodo impara I lo inoltra e poi passa a stato **DONE**, così non invia più messaggi.*

- Stati $S = \text{INITIATOR, SLEEPING, DONE}$
- Informazione *I*

Entità INITIATOR

```
quando parte:
    manda I a tutti i vicini
    diventa DONE

se riceve I:
    ignora
```

Entità SLEEPING

```
quando parte:
    dorme

se riceve I:
    manda I a tutti i vicini
    diventa DONE
```

Entità DONE

```
qualunque cosa succeda:  
  ignora
```

Questo porta a due osservazioni fondamentali: - esiste **local termination** (i nodi finiscono in istanti diversi quando diventano DONE); - non esiste **global termination** “conosciuta”: nessun nodo sa quando l’intero processo è finito → **termination detection problem**.

Protocollo FLOODING (broadcast “standard”)

Ottimizzazione: se un nodo riceve I da `sender`, non serve rimandarglielo indietro; grazie alla **local orientation** posso distinguere il mittente e inviare a $N(x) \setminus sender$.

- Stati $S = \text{INITIATOR, SLEEPING, DONE}$
- Informazione I

Entità INITIATOR

```
quando parte:  
  manda I a tutti i vicini  
  diventa DONE  
  
se riceve I:  
  ignora
```

Entità SLEEPING

```
se riceve I da `sender`:  
  per ogni vicino  $y \in N(x)$ :  
    se  $y \neq sender$ :  
      manda I a  $y$   
  diventa DONE
```

Entità DONE

```
qualunque cosa succeda:  
  ignora
```

Correttezza (schema di dimostrazione “da esame”)

Tutti ricevono I

Sotto CN (Connectivity) e TR (Total Reliability), esiste un cammino dal INITIATOR a qualunque nodo. Il primo nodo sul cammino che riceve I lo inoltra (una sola volta) ai suoi vicini “in avanti”, e così via: per induzione lungo il cammino, ogni nodo viene raggiunto e riceve I .

Terminazione

Ogni nodo (a parte INITIATOR) quando riceve I esegue **una sola** azione di invio e poi passa a DONE; in DONE non invia più nulla. Quindi ogni nodo termina localmente.

Calcolo della complessità

Notazione

- Grafo $G = (V, E)$
- $n = |V|$ numero di nodi
- $m = |E|$ numero di archi
- $N(x)$ = insieme dei vicini di x
- INITIATOR s
- Diametro $D(G)$

Misure di complessità nel distribuito: messaggi vs tempo

Nel distribuito si misurano soprattutto:

1. **Amount of communication**: numero di messaggi (o bit) scambiati.
2. **Tempo**: dipende dal **modello temporale** assunto.
 - **Total synchrony / Ideal time**: 1 unità di tempo per trasmettere 1 messaggio (ragionamento “a hop/round”).
 - **Asynchrony / Casual time**: clock non sincronizzati e ritardi imprevedibili; si misura il tempo come **lunghezza della longest message chain** tra tutte le possibili esecuzioni.

Nota importante: **la message complexity non dipende** dal fatto che il sistema sia sincrono o asincrono; la **time complexity** sì, perché cambia la nozione di “quanto dura” un’esecuzione.

Message complexity di FLOODING

Facendo una stima banale, potremmo dire che dato che ogni nodo riceve, invia e poi si ferma: in ogni arco passeranno al massimo 2 messaggi. Quindi potremmo stimare che i messaggi inviati siano $2m \in O(m)$.

Però il nostro algoritmo è più preciso in realtà, quindi potremmo fare un'analisi più attenta per calcolare in modo più preciso la complessità legata ai messaggi.

Passo 1 — Conta dei messaggi inviati dal INITIATOR

L'initiator s invia il messaggio una sola volta a tutti i suoi vicini:

$$\text{msg inviati da } s = |N(s)|$$

Passo 2 — Conta dei messaggi inviati dagli altri nodi

Ogni nodo $x \neq s$ quando riceve I lo inoltra a tutti i vicini **tranne** il `sender`.

Quindi invia esattamente $|N(x)| - 1$ messaggi:

$$\text{msg inviati da } x \neq s = |N(x)| - 1$$

Passo 3 — Somma totale

Sommo su tutti i nodi:

$$M(\text{Flooding}(G)) = |N(s)| + \sum_{x \neq s} (|N(x)| - 1)$$

Sviluppo:

$$M(\text{Flooding}(G)) = |N(s)| + \sum_{x \neq s} |N(x)| - \sum_{x \neq s} 1 = \sum_x |N(x)| - (n - 1)$$

Passo 4 — Uso dell'identità $\sum_x |N(x)| = 2m$ (grafo non orientato)

In un grafo con link bidirezionali (non orientato), la somma dei gradi vale:

$$\sum_x |N(x)| = 2m$$

Sostituendo:

$$M(\text{Flooding}(G)) = 2m - (n - 1) = 2m - n + 1$$

Conclusione:

$$M(\text{Flooding}(G)) = 2m - n + 1 \in O(m)$$

Motivazione: la somma dei gradi è $2m$ perché se ci pensi in un grafo connesso quando calcoli il grado di tutti i nodi, in realtà stai contando ogni arco due volte. Infatti ogni arco deve

essere conteggiato e viene conteggiato 2 volte perché ha due nodi collegati

Time complexity di FLOODING
perché è $\Theta(D(G))$

Definizioni

- Distanza minima in un cammino da a a b

$$d(a, b)$$

- Eccentricità o *Radius* (distanza minima tra a e il nodo più lontano y):

$$r(a) = \max_y d(a, y)$$

- Diametro (distanza tra i nodi più lontani tra loro nel grafo):

$$D(G) = \max_a r(a) = \max_{x,y} d(x, y)$$

A) Tempo *sincrono = Ideal time (total synchrony)

Assunzione temporale: “1 unità di tempo = 1 hop”, cioè in una unità ogni vicino riceve il messaggio (ragionamento a round).

Upper bound per FLOODING

tempo impiegato dal INITIATOR a raggiungere tutti

In ideal time (1 unità per hop), dopo t unità hanno ricevuto I tutti i nodi a distanza $\leq t$ dal INITIATOR. Quindi il tempo dell'esecuzione con initiator s è:

$$T_s \leq r(s) = \max_y d(s, y)$$

Nel worst-case su tutte le scelte possibili di initiator:

$$T(\text{Flooding}) \leq \max_s r(s) = D(G)$$

Lower bound per qualsiasi broadcast (non solo flooding)

Nel caso peggiore l'initiator può essere un nodo “estremo”, e qualche nodo può essere a distanza $D(G)$. Quindi nessun algoritmo può fare meglio di:

$$T(\text{Broadcast}(G)) \geq \max_{x,y} d(x,y) = D(G)$$

Conclusione (tight bound):

$$T(\text{Flooding}) = \Theta(D(G))$$

N.B. : il diametro di un grafo può essere massimo $n - 1$. Se ci pensi per raggiungere il nodo più lontano dovrai al massimo attraversare tutti gli altri nodi del grafo.

B) Tempo asincrono = Casual time (longest message chain)

Assunzione temporale: clock non sincronizzati, ritardi imprevedibili ma finiti; non ha senso parlare di round globali.

La misura consigliata nelle slide è la **lunghezza della longest message chain** (dipendenza causale) nel worst-case tra tutte le esecuzioni possibili.

Per FLOODING, la catena causale che “porta” I al nodo più lontano dall’initiator deve attraversare almeno $d(s, y)$ hop; nel worst-case:

$$T^{async}(\text{Flooding}) \in \Theta(D(G))$$

dove però l’interpretazione è: **numero di messaggi in catena causale**, non “unità di tempo fisiche”.

■ In pratica: **stesso ordine** $\Theta(D(G))$, ma cambia il significato:

- ■ sincrono: “ $D(G)$ round (1 hop per round)”
 - ■ asincrono: “ $D(G)$ dipendenze causali minime lungo il cammino peggiore”
-

Lower bound sui messaggi per Broadcast: da $n - 1$ a m

Bound “ovvio”: almeno $n - 1$

Alla fine, tutti i $n - 1$ nodi diversi dall’initiator devono ricevere l’informazione almeno una volta:

$$M(\text{Bcast}) \geq n - 1$$

Bound più forte: almeno m (idea dell’arco “non usato”)

Si può mostrare che in un’esecuzione corretta deve passare **almeno un messaggio su ogni link** nel worst-case, quindi:

$$M(\text{Bcast}) \geq m$$

Passaggi dell’argomento (versione Santoro): 1. Supponi esista un protocollo corretto che usa $< m$ messaggi su un grafo G . 2. Allora esiste almeno un arco $e = (x, y)$ su cui **non passa mai alcun messaggio** in quell’esecuzione. 3. Costruisci un nuovo grafo G' rimuovendo e e aggiungendo un nuovo nodo z collegato a x e y

. 4. Ripeti “la stessa esecuzione” su G' : siccome su e non passava nulla, x e y non hanno motivo di inviare a z . 5. Quindi z non riceve mai $I \rightarrow$ contraddizione con la correttezza del broadcast.

Conclusion: qualsiasi broadcast richiede $\Omega(m)$ messaggi.

Risultato finale

Per broadcast sotto le restrizioni standard (e unique initiator), e per FLOODING: -

$$M(\text{Flooding}(G)) = 2m - n + 1 \in \Theta(m)$$

- $T(\text{Flooding}(G)) \in \Theta(D(G))$
 - $T^{\text{ideal}}(\text{Flooding}) = \Theta(D(G))$
 - $T^{\text{asyn}}(\text{Flooding}) = \Theta(D(G))$ (misurato come longest message chain)

Possiamo fare meglio di così?

Facciamo queste due considerazioni: - Message complexity minima con un albero: - In un albero il numero di archi è $m = n - 1$ - $M(\text{Flooding}(G)) = 2m - n + 1 = 2(n - 1) - n + 1 = n - 1 = m$

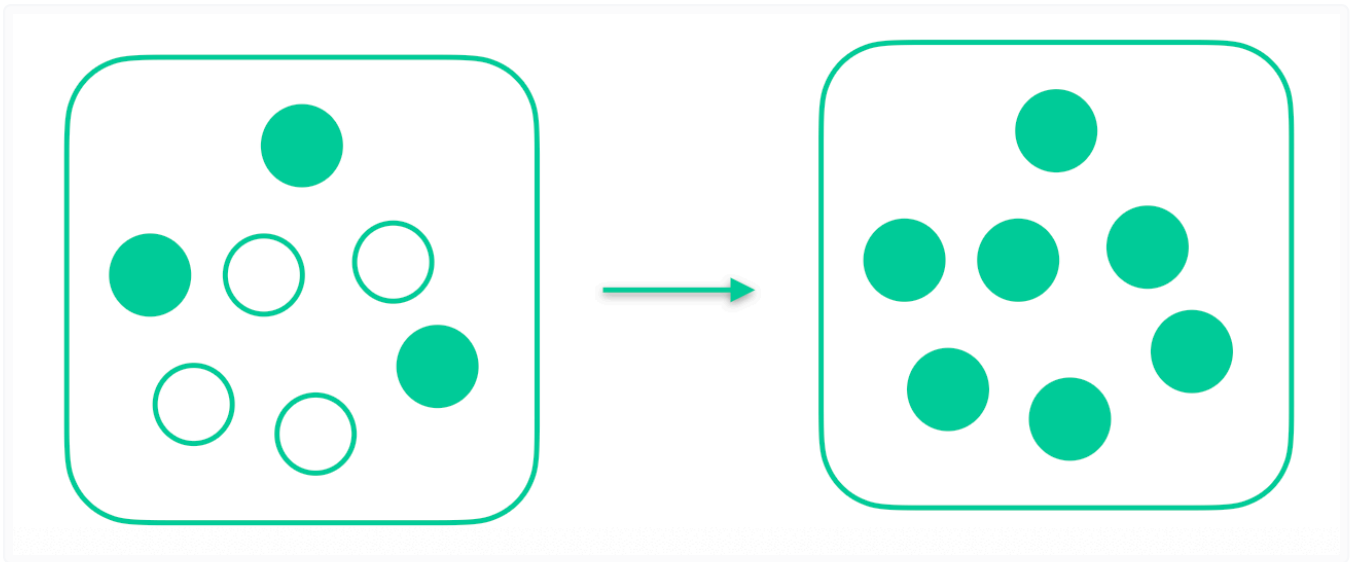
- Message complexity massima con un grafo connesso:
 - In un grafo connesso il numero di archi è $m = \binom{n}{2} = \frac{n(n-1)}{2}$
 - Quindi $M(\text{Flooding}(G)) \in O(n^2)$

Ma dobbiamo per forza usare il flooding asincrono in un grafo connesso?

Con un broadcast classico avremmo: - $M(\text{SimpleBroadcast}) = n - 1$ - $T^{\text{ideal}}(\text{SimpleBroadcast}) = 1$

Infatti il flooding è efficiente in un albero, ma non è adatto per i grafi generici.

Wake-up problem



Il **wake-up problem** è la “versione più generale” del broadcast: devi far partire una computazione che coinvolga *tutti* i nodi, ma **all’inizio solo alcuni sono attivi** (possono iniziare a mandare messaggi), mentre gli altri sono **ASLEEP** e si “svegliano” solo quando ricevono un messaggio.

- Situazione: un task deve coinvolgere **tutte** le entità, ma inizialmente **solo alcune** sono attive (*awake*) e le altre sono inattive (*asleep*).
- Obiettivo: **svegliare tutti i nodi** (portarli nello stato AWAKE).
- Relazione con Broadcast:
 - **Broadcast = Wake-up con un solo initiator** (un solo nodo inizialmente awake).
 - **Wake-up = Broadcast con più initiator** (più nodi inizialmente awake).

Modello (stati e restrizioni)

- Stati:
- $S = \text{ASLEEP}, \text{AWAKE}$
- Stati iniziali:
- $S_{init} = \text{ASLEEP}$ (tutti asleep, ma alcuni possono svegliarsi spontaneamente)
- Stato terminale:
- $S_{term} = \text{AWAKE}$
- Restrizioni: quelle “standard” del modello usato nel corso (nelle note: “R”).

Protocollo WFlood (wake-up via flooding)

Obiettivo Tutti i nodi che siano in stato

AWAKE (sveglio)

- Stati $S = \text{ASLEEP}, \text{AWAKE}$
- Stato iniziale $S_{init} = \text{ASLEEP}$ (tutti asleep, ma alcuni possono svegliarsi spontaneamente)
- Stato terminale $S_{term} = \text{AWAKE}$
- Messaggio W (richiesta di svegliarsi)

Usiamo la **stessa strategia del flooding** del broadcast risolve anche il wake-up.

Entità ASLEEP

```
quando parte:
    manda W a tutti i vicini
    diventa AWAKE

se riceve W da `sender`:
    per ogni vicino  $y \in N(x)$ :
        se  $y \neq \text{sender}$ :
            manda W a  $y$ 
    diventa AWAKE
```

Complessità in messaggi (message complexity)

- Caso 1 initiator: coincide col flooding del broadcast
 - $M = 2m - n + 1$
- Caso generale con k initiator:
 - $M = 2m - (n - k)$
- Caso tutti n initiator:
 - $M = 2m$

Come si legge $2m - (n - k)$?

- $2m$ è il conteggio “grezzo”: in un grafo non orientato ogni arco può essere percorso al massimo **due volte** (una per direzione).
- Però **non tutti i nodi fanno l'azione “receiving(W) then forward”**: se un nodo è un iniziatore, si sveglia per spontaneous impulse e **non ha un “sender” da escludere** in quel primo step (di fatto “manca” un inoltro rispetto al caso in cui si svegliasse ricevendo).
- Ci sono k iniziatori, quindi i nodi che *non* sono iniziatori sono $n - k$ e sono proprio quelli che generano quel tipo di inoltro “da ricezione” con esclusione del sender: il termine $(n - k)$ è la correzione che compare nelle slide.

Bound generali

- Per WFlood vale:

$$2m \geq M[\text{WFlood}] \geq 2m - n + 1$$

- Quindi in ordine di grandezza:

$$M(\text{Wake-Up}/R) \in \Theta(m)$$

Caso speciale: Tree - Se G è un albero e k' è il numero di initiator:

$$M[\text{WFlood}/\text{Tree}] = n + k' - 2$$

Complessità in tempo (ideal time / time complexity)

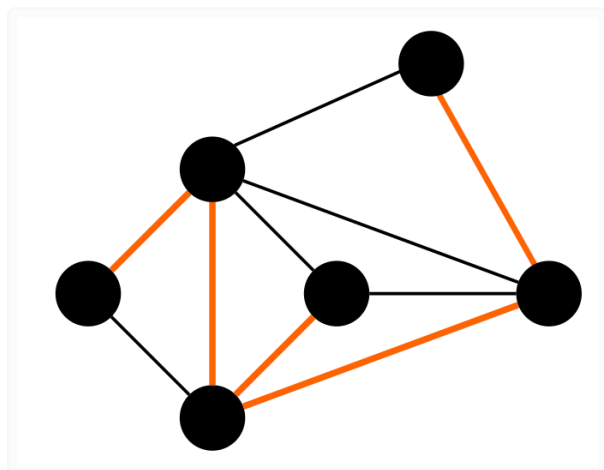
Intuizione: rispetto al broadcast, in generale è **non maggiore** perché un nodo può essere più vicino a *qualche* initiator.

Worst case (bound di ordine):

$$T(\text{Wake-Up}/R) \in \Theta(d)$$

dove d è il diametro del grafo (come nelle note “ideal time”). - Caso $k = 1$ (un solo initiator): il tempo coincide con Flooding, tipicamente $O(D)$.

Spanning Tree



Uno **spanning tree** (in italiano: *albero di copertura*) di un

grafo connesso $G = (V, E)$ è un **sottografo** $T = (V, E')$ tale che:

1. contiene **tutti** i vertici di G (quindi “spanning” = *che copre/spazia* su tutti i nodi):

$$V(T) = V(G)$$

2. usa solo alcuni archi del grafo originale:

$$E' \subseteq E$$

3. è un **albero**, cioè è **connesso e aciclico** (non contiene cicli).

Proprietà super importante Se G è connesso e ha $n = |V|$ nodi, allora **ogni spanning tree ha esattamente $n - 1$ archi**. È la proprietà classica degli alberi; nel testo CLRS viene richiamata quando parla di spanning tree come sottoinsieme aciclico che connette tutti i vertici.

Nei protocolli distribuiti spesso si costruisce prima uno spanning tree e poi si eseguono broadcast/traversal **solo sull'albero** per ridurre i messaggi: sull'albero, ad esempio, il broadcast costa esattamente $n - 1$ messaggi.

Cosa “sanno” i nodi a fine costruzione Nel problema **SPT** (spanning tree construction) in ambiente distribuito, alla fine non è richiesto che un nodo conosca tutto T : ogni nodo x deve selezionare localmente un insieme $\text{Tree-neighbors}(x) \subseteq N(x)$ che rappresenta **quali vicini sono collegati a lui nell'albero**.

Spanning Tree Construction (SPT)

Problema e output “locale”

Dato un grafo connesso $G = (V, E)$, vogliamo costruire uno **spanning tree** $T = (V, E')$ con $E' \subseteq E$ e T aciclico e connesso. Nella versione distribuita, **alla fine ogni nodo non conosce tutto l'albero**, ma solo quali tra i suoi vicini sono collegati a lui tramite un arco di T (variabile tipo `Tree-neighbours(x)`).

Assunzioni: - **Single initiator**, link bidirezionali, affidabilità totale, grafo connesso.

Idea 1: costruire lo spanning tree “da broadcast”

C'è un fatto generale molto utile: **l'esecuzione di un qualunque protocollo di broadcast** (con unico iniziatore) **induce uno spanning tree**: per ogni nodo $x \neq s$, definisci `parent(x)` come il vicino da cui x riceve l'informazione **per la prima volta**; la relazione “parent” definisce un albero radicato nell'iniziatore.

Ma attenzione: sapere solo `parent(x)` **non basta** per risolvere SPT come definito nel corso/libro, perché ogni nodo deve anche determinare **chi sono i figli** e quali vicini **non** sono tree-neighbors; questo richiede ulteriore “feedback” (es. messaggi YES/NO).

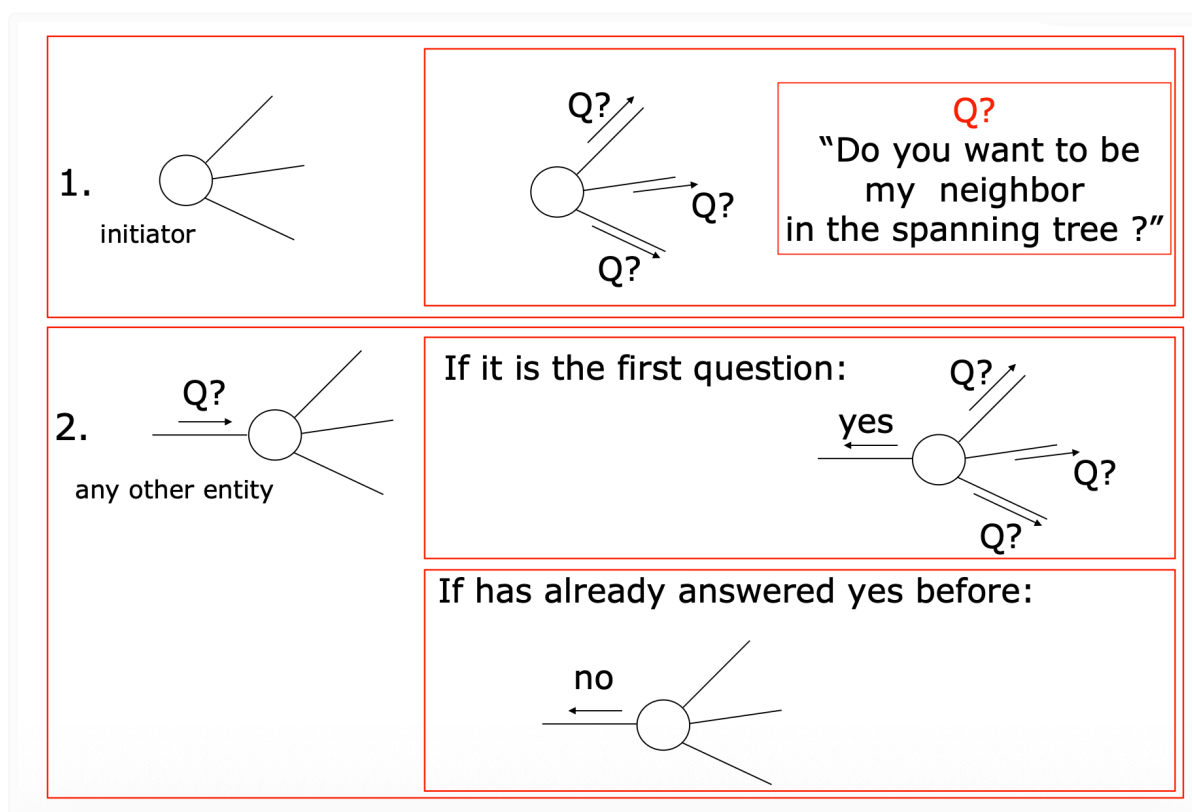
Protocollo SHOUT

costruzione per “richieste Q ” + risposte YES/NO

Intuizione: l'iniziatore chiede ai vicini di diventare suoi vicini nell'albero; ogni nodo dice **YES** solo alla **prima** richiesta che riceve (scegliendo così il proprio parent) e risponde **NO** alle richieste successive.

Stati: - $S = \text{INITIATOR, IDLE, ACTIVE, DONE}$ - $S_{init} = \text{INITIATOR}$, IDLE - $S_{term} = \text{DONE}$

Messaggi: - Q richiesta di collegamento - YES/NO risposta



Variabili tipiche: - `parent`, `root`, `Tree-neighbours(x)` (insieme dei vicini in T), `counter` (conta le risposte ricevute dai vicini).

Correttezza (idea chiave): - Ogni nodo (tranne l'iniziatore) manda **esattamente un YES** $\Rightarrow$ ogni nodo sceglie un solo parent. - La relazione costruita da `Tree-neighbours` definisce un **albero connesso** che contiene tutti i nodi (terminazione locale).

Pseudocodice

Entità INITIATOR

```
quando parte:
  root  $\leftarrow$  true
  TreeNeighbours  $\leftarrow \emptyset$ 
  send Q to N(x)
  counter  $\leftarrow$  0
  status  $\leftarrow$  ACTIVE
```

Entità IDLE

```
riceve Q da sender:
  root  $\leftarrow$  false
  parent  $\leftarrow$  sender
  TreeNeighbours  $\leftarrow$  {sender}
  send YES to parent
  counter  $\leftarrow$  1
  if counter = |N(x)| then
    status  $\leftarrow$  DONE
  else
    send Q to N(x) \ {sender}
    status  $\leftarrow$  ACTIVE
```

Entità ACTIVE

```
riceve Q da sender:
  send NO to sender

riceve YES da sender:
  TreeNeighbours  $\leftarrow$  TreeNeighbours  $\cup$  {sender}
  counter  $\leftarrow$  counter + 1
  if counter = |N(x)| then status  $\leftarrow$  DONE

riceve NO da sender:
  counter  $\leftarrow$  counter + 1
  if counter = |N(x)| then status  $\leftarrow$  DONE
```

Calcolo complessità messaggi

Notazione

- Grafo $G = (V, E)$
- $n = |V|$ numero di nodi
- $m = |E|$ numero di archi

- $N(x)$ = insieme dei vicini di x
- INITIATOR s
- Diametro $D(G)$

Intanto partiamo da quello che abbiamo già analizzato: sappiamo che il **Flooding (broadcast)** prevede $2m - (n - 1)$ messaggi.

Interpretazione intuitiva: in ogni arco passerebbero due messaggi se ogni nodo mandasse a tutti i suoi vicini il messaggio “ $2m$ ”, ma nel flooding ottimizzato **ogni nodo non-initiator** “risparmi” 1 messaggio perché **non rispedisce al sender** $\Rightarrow$ risparmi $n - 1$.

SHOUT = FLOODING + REPLY

Analizzando il caso dello SHOUT dobbiamo capire quanti messaggi vengono mandati per ogni tipo: **Q, YES, NO**.

Messaggi Q

Questi messaggi vengono inviati nella fase di flooding. Facciamo l'analisi di questi messaggi in base al tipo di arco (*tree vs non-tree*).

Nel *worst case* distinguiamo i casi:

- **Q-YES**, dove la richiesta di entrare nell'albero è accettata (tree).
 - Questi casi si verificano negli archi dello spanning tree: $n - 1$.
- **Q-Q**, negli archi non-tree possono passare 2 messaggi Q dai due nodi (INITIATOR, o appena entrati in IDLE).
 - Gli archi non appartenenti all'albero sono: $m - (n - 1)$
 - **2 Q per arco** $\Rightarrow 2[m - (n - 1)]$.

Quindi in totale abbiamo:

$$Q_{tot} = (n - 1) + 2[m - (n - 1)] = 2m - n + 1$$

Messaggi NO

Sfruttiamo l'analisi fatta prima, i messaggi di NO si presentano nel caso in cui un nodo non sia entrato nello spanning tree, quindi la quantità di messaggi è uguale a Q - Q:

$$NO_{tot} = 2m - n + 1$$

Messaggi YES

Anche qui sfruttiamo l'analisi fatta prima, i messaggi di YES si presentano solo nel caso in cui un nodo sia entrato nello spanning tree, quindi:

$$YES_{tot} = n - 1$$

Sommando tutto

$$M(\text{SHOUT}) = Q_{tot} + NO_{tot} + YES_{tot}$$

tra cui

$$M(\text{SHOUT}) = (2m - n + 1) + 2[m - (n - 1)] + (n - 1)$$

Semplificando diventa:

$$M(\text{SHOUT}) = (2m - n + 1) + (2m - 2n + 2) + (n - 1) = 4m - 2n + 2$$

Quindi abbiamo

$$M(\text{SHOUT}) = 4m - 2n + 2 = 2 \cdot (2m - n + 1).$$

In confronto al flooding abbiamo il doppio dei messaggi:

$$M(\text{SHOUT}) = 2(2m - n + 1) = 2 \cdot M(\text{Flooding})$$

Possiamo ottimizzare?

Protocollo SHOUT+ (ottimizzazione: senza NO)

I messaggi NO sono superflui e potremmo dedurli: quando un nodo ACTIVE riceve una richiesta Q , la interpreta implicitamente come “NO” (incrementa `counter` e basta). In questo modo: - su ogni link transitano esattamente **due messaggi** (o Q -YES oppure Q - Q), quindi:

$$M(\text{SHOUT}^+) = 2m.$$

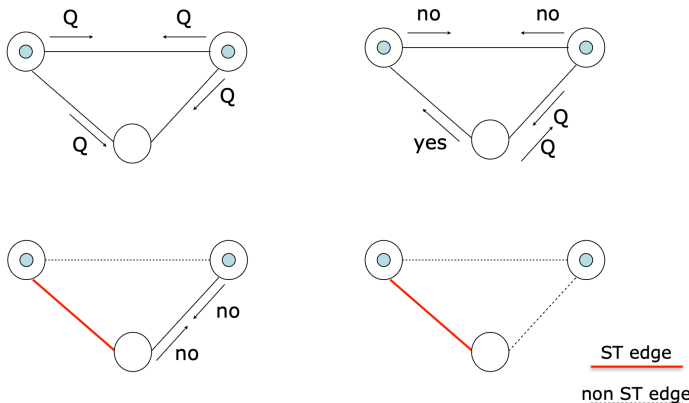
Entità ACTIVE

```
riceve Q da sender:
  counter ← counter + 1
  if counter = |N(x)| then status ← DONE

riceve YES da sender:
  TreeNeighbours ← TreeNeighbours ∪ {sender}
  counter ← counter + 1
  if counter = |N(x)| then status ← DONE
```

Importante: cosa succede con più iniziatori?

What happens with SHOUT if there are multiple initiators ?



Se toglie l'assunzione "single initiator", i

protocolli progettati per unico iniziatore **possono fallire**: ad esempio SHOUT con due initiator può costruire una **foresta** (non connessa).

Risultato teorico (libro): **SPT è deterministically unsolvable sotto le sole restrizioni standard R** (cioè senza imporre un unico iniziatore o assunzioni extra).

Spanning Tree Construction by Traversal (Depth-First Traversal)

Un'altra famiglia di soluzioni costruisce lo spanning tree tramite una **visita DFS distribuita** usando un **token** (Forward/Return/Back-edge). - Quando un nodo riceve il **ForwardToken** per la prima volta, memorizza chi lo ha inviato (quello è il **parent**) e prova a inoltrare il token a un vicino non visitato; se un nodo riceve di nuovo il ForwardToken su un arco, risponde con **Back-edge token** (quell'arco non è nell'albero). - Eliminando i back-edge si ottiene lo spanning tree; **parent(x)** è chi ha inviato per primo il token, e i figli sono i vicini che non risultano back-edge.

(Questa parte è utile per capire che esistono costruzioni SPT anche "per traversal", ma in genere SHOUT/SHOUT+ sono più puliti come costruzione locale dei **Tree-neighbours**.)

Spanning Tree Construction by Traversal

Depth-First Traversal (DFT)

Obiettivo e idea di base

Dato un grafo di comunicazione connesso $G = (V, E)$, vogliamo costruire uno **spanning tree** usando una **traversal** (visita) che renda i nodi **visited in modo sequenziale** (uno alla volta). L'algoritmo parte da un **initiator** già visited, mentre tutti gli altri sono inizialmente unvisited.

Una traversal distribuita usa un messaggio speciale chiamato **token**: quando un nodo riceve il token, viene considerato **visited**. Se la traversal è **depth-first**, gli archi usati “alla prima visita” formano uno spanning tree.

Intuizione DFS: “si prova a mandare avanti il token il più possibile” (profondità), e si torna indietro solo quando non ci sono più vicini utili.

Restrizioni (assunzioni del modello)

- **Single initiator**
 - **Bidirectional links**
 - **Connectivity**
 - **Total reliability**
-

Messaggi (tipi di token)

- `ForwardToken` : esplora un vicino (passo “in avanti” DFS)
 - `ReturnToken` : ritorna al parent dopo aver finito un sottoalbero
 - `BackEdgeToken` : segnala che l’arco usato porta a un nodo già visited (quindi è una **back-edge**, non fa parte dell’albero)
-

Spanning tree ottenuto (root/parent/children)

Dopo l’esecuzione: - **Root** = l’**initiator** - **Parent di x** = il nodo da cui x ha ricevuto **per la prima volta** il token (il suo `entry`) - **Children di x** = i vicini di x che **non** risultano collegati a x da una back-edge (in pratica: i vicini effettivamente esplorati come figli nel DFS)

Stato locale e variabili

Stati

$S = \text{INITIATOR, IDLE, VISITED, DONE}$

- Stati iniziali possibili: $S_{init} = \text{INITIATOR, IDLE}$ - Stato terminale: $S_{term} = \text{DONE}$

Variabili locali per ogni nodo x

- `entry` : il vicino da cui x ha ricevuto il **primo** `ForwardToken` (definisce il **parent**)
- `Unvisited` : insieme dei vicini “ancora da provare”
- iniziatore: `Unvisited := N(x)`
- nodo non iniziatore, alla prima ricezione: `Unvisited := N(x) \setminus \{\text{sender}\}`

- `initiator`: booleano per distinguere la radice

Operazione ausiliaria: - `pick(Unvisited)`: estrae un elemento dall'insieme e lo rimuove (così non lo riprovi).

Protocollo base: DFT (Depth-First Traversal)

Procedura comune `VISIT`

Idea: se ho ancora vicini non provati, mando il token a **uno** di loro; altrimenti torno al parent.

```
procedure VISIT:
  if |Unvisited| > 0 then
    next := pick(Unvisited)
    send ForwardToken to next
    diventa VISITED
  else
    if not(initiator) then
      send ReturnToken to entry

    diventa DONE // terminazione globale (se si tratta della radice)
```

INITIATOR

```
quando parte:
  Unvisited := N(x)
  initiator := true
  esegue VISIT
```

IDLE

```
on receiving ForwardToken from sender:
  entry := sender
  Unvisited := N(x) \ {sender}
  initiator := false
  esegue VISIT
```

VISITED

```
on receiving ForwardToken from sender:
  Unvisited := Unvisited \ {sender}
  send BackEdgeToken to sender

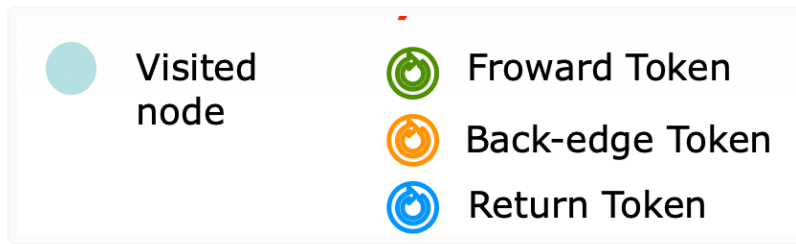
on receiving ReturnToken:
  esegue VISIT

on receiving BackEdgeToken:
  esegue VISIT
```

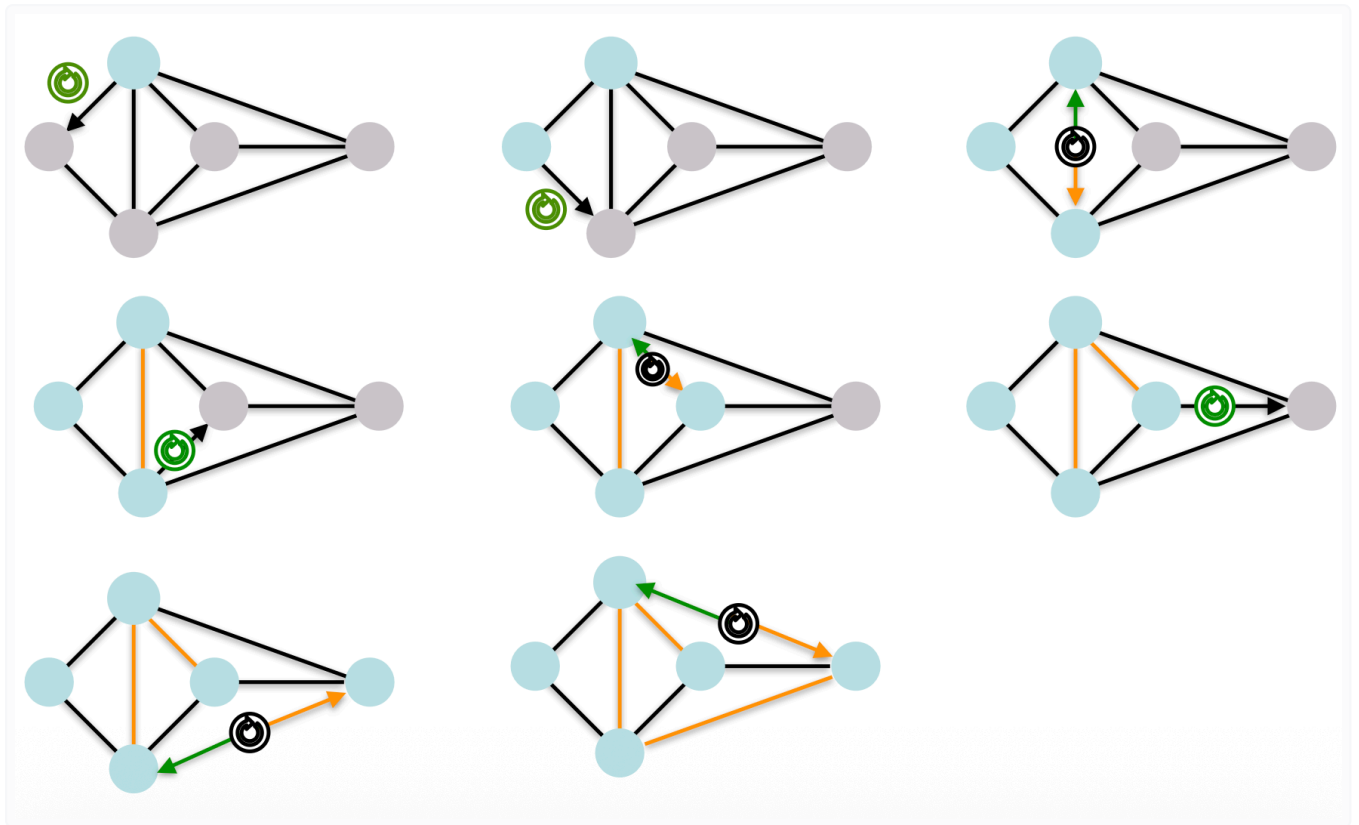
Quando un nodo è DONE, non riceve più messaggi (ha concluso)

Esempio grafico

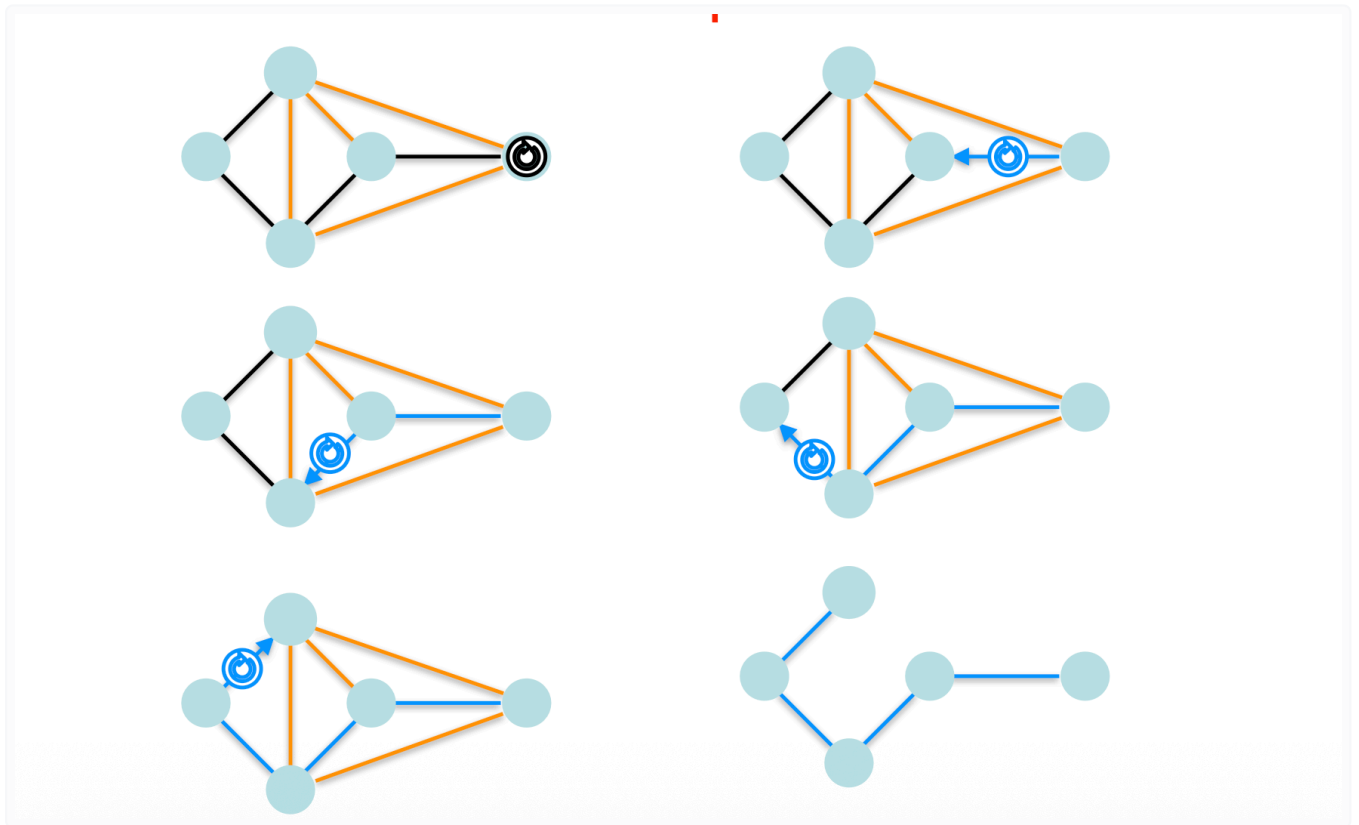
Legenda



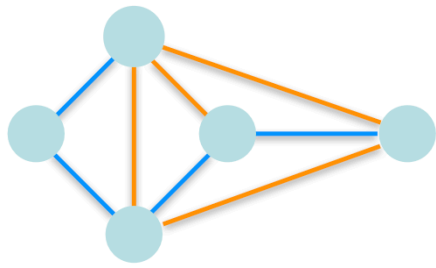
La root inizia a mandare il Forward Token



A questo punto l'ultimo nodo ha finito i nodi a cui mandare il Forward Token, quindi manda il Return Token



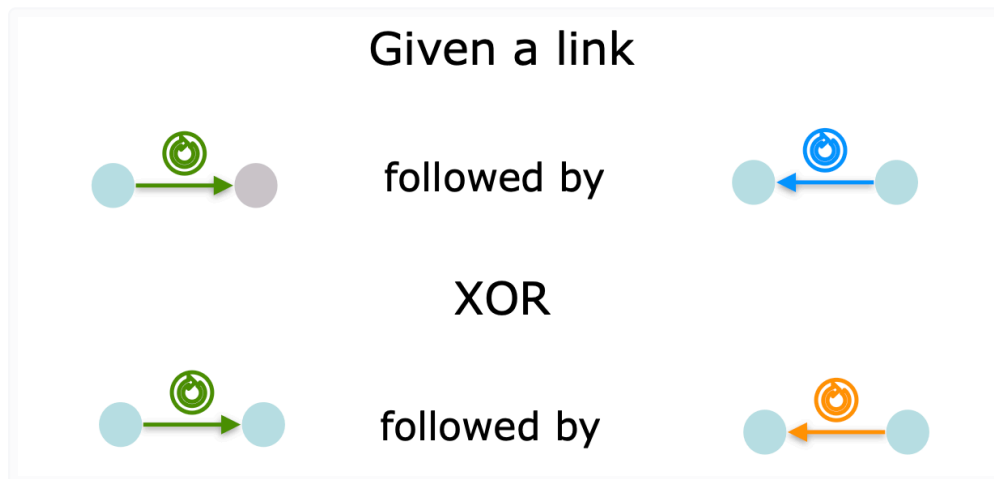
Quando il Return Token arriva alla root, abbiamo lo spanning tree del grafo.



Removing back-edges
we have a spanning tree

Complessità del protocollo base DFT

Message complexity



Dato che ad ogni Forward corrisponde una risposta con un Return o Backedge, allora sono 2 i messaggi per ogni arco. Ogni link viene “coinvolto” in modo tale che il totale risulta:

$$\text{Message}(\text{DFT}) = 2m$$

e non si può migliorare in ordine di grandezza:

$$\text{Message}(\text{DFT}(G)) \in \Omega(m)$$

(l’idea della dimostrazione è analoga al lower bound del broadcast).

Time complexity

La traversal richiede che **un solo nodo alla volta** possieda il token (l’idea di traversal è proprio “passare il token sequenzialmente”). Quindi **non puoi avere due token in giro**, e non puoi esplorare due rami in parallelo.

Quanti “passi” servono? (catena più lunga)

Nel protocollo DF Traversal (DFT) su ogni link (x, y) succede:

- se x manda T (ForwardToken) a y , allora y risponde a x con **uno** tra Return oppure Backedge.

Quindi **su ogni link passano esattamente 2 messaggi**.

Totale messaggi = $2m$ e siccome **sono inviati in sequenza (uno dopo l'altro)**, la catena temporale più lunga è lunga $2m$:

$$T[\text{DFT}] = 2m.$$

Le slide lo dicono esplicitamente: “Since traversal is sequential, and $2m$ messages are sent sequentially... TIME COMPLEXITY = $2m$ ”.

Il libro fa la stessa osservazione: “Since the traversal is sequential, $T = M$; hence $T = M = 2m$.”

Intuizione: in un grafo denso (molti archi), la DFS “perde tempo” a tentare vicini che poi risultano back-edge: ogni tentativo costa comunque un messaggio T + risposta (Backedge), e tutto avviene in coda, uno alla volta.

Lower bound: perché non puoi fare meno di $n - 1$

Anche se potessi eliminare ogni spreco, devi comunque far sì che ogni nodo venga visitato almeno una volta, in modo sequenziale:

$$T(\text{DFT}(G)) \geq n - 1.$$

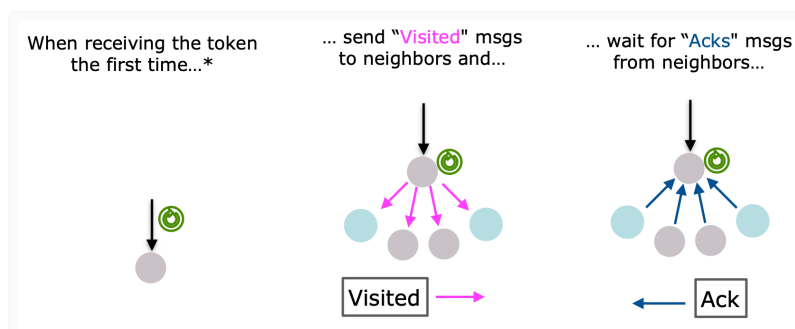
Miglioramento: evitare di mandare il token sulle back-edges (Visited/Ack)

Problema del protocollo base

Nel protocollo base, un nodo può ricevere il token “inutilmente” da più vicini (tentativi di visita che diventano back-edge). L'idea è **prevenire** questi tentativi: far sapere ai vicini “sono già visited” prima che provino a mandarmi il token.

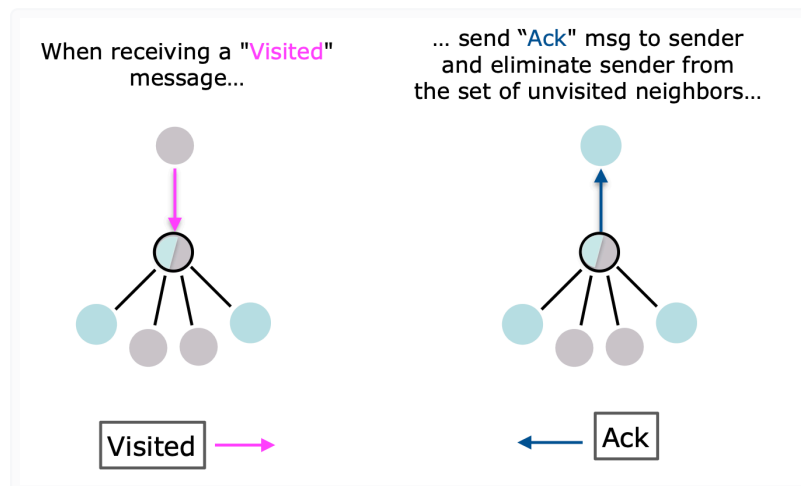
Idea (Visited/Ack)

Quando un nodo riceve il token **la prima volta** (vale anche per l'initiator):



1. manda Visited a tutti i vicini
2. aspetta un Ack da ciascun vicino
3. poi procede come prima, ma manda ForwardToken **solo** a vicini che risultano non visited

Quando un nodo riceve un messaggio Visited:



- risponde con Ack - elimina il sender dal

proprio insieme Unvisited (così non proverà più a visitarlo col token)

Differenza concettuale: DFT normale vs DFT con Visited/Ack

DFT normale (con BackedgeToken)

- Il token “scopre” che un arco è back-edge **solo quando prova ad attraversarlo**.
- Quindi un nodo può ricevere ForwardToken anche **da tanti vicini diversi** (tutti i tentativi diventano back-edge), e ciascun tentativo costa tempo *sequenziale*.

DFT ottimizzata (Visited/Ack)

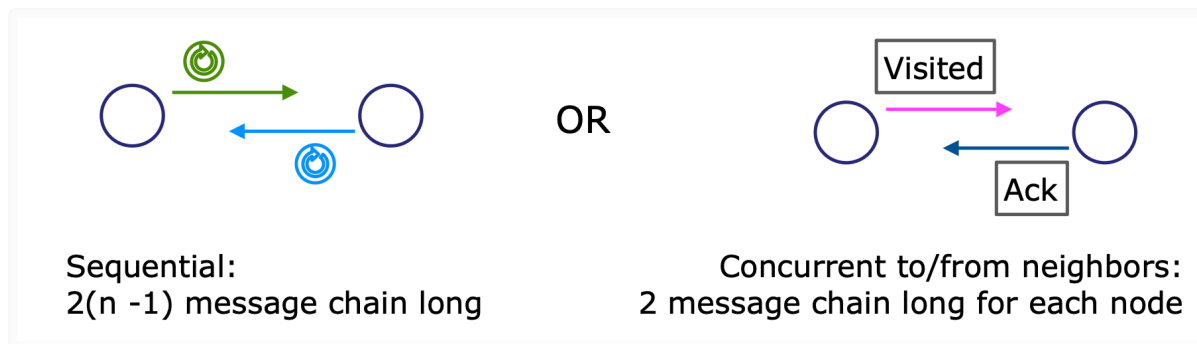
Le slide dicono: - quando un nodo riceve il token **la prima volta**, manda Visited ai vicini e aspetta Ack - quando un nodo riceve Visited, risponde Ack ed elimina il sender da Unvisited

Effetto: i back-edge vengono “scoperti” *prima* che il token ci passi sopra. Quindi il token verrà mandato **solo** lungo archi che portano davvero a nodi ancora non visitati (i futuri archi dell’albero).

In pratica: la DFT normale “paga” i back-edge **in tempo**, la versione Visited/Ack li paga **in messaggi extra ma paralleli**.

Time complexity del miglioramento

Le slide scompongono il tempo in due blocchi: **Token lungo l’albero (sequenziale)** e **Handshake Visited/Ack (parallelo)**



La parte “token lungo l’albero” resta sequenziale: catena lunga $2(n - 1)$.

Se eviti di mandare il token sui back-edge, il token attraversa solo gli archi dell’albero DFS costruito.

Un albero ha $n - 1$ archi, e su ciascun arco dell’albero il token fa “andata e ritorno” (Forward + Return), quindi:

$$2(n - 1)$$

messaggi in **catena sequenziale**.

Gli scambi **Visited/Ack** ($2n$) verso i vicini avvengono in parallelo per ogni nodo (catena lunga 2 per nodo), e il contributo massimo complessivo porta alla stima:

$$2(n - 1) + 2n = 4n - 2$$

quindi:

$$\text{Time} \in O(n)$$

Intuizione decisiva: con Visited/Ack, il costo che prima dipendeva da m (tentativi su tanti archi) viene “spostato” in una fase locale parallela; la parte davvero sequenziale resta legata alla dimensione dell’albero ($n - 1$), non alla densità del grafo.

Message complexity del miglioramento

Conteggio come nelle slide:

- per ogni arco dell’albero: 2 messaggi di token
- per ogni arco non d’albero: fino a 4 messaggi (Visited/Ack in entrambe le direzioni, concettualmente)

Totale:

$$2(n - 1) + 2(n - 1) + 4(m - (n - 1)) = 4m$$

quindi:

$$\text{Message} \in O(m)$$

TODO: da qui

Confronto qualitativo con SHOUT+ e nota sul diametro

- $\text{Message}(\text{SHOUT}+) = 2m$
- $\text{Message}(\text{DFT}) = 2m$ (più eventuali overhead se usi l'ottimizzazione con Visited/Ack, che porta a un costante maggiore)

Punto importante: tecniche diverse costruiscono spanning tree diversi; inoltre, lo stesso protocollo sullo stesso grafo può produrre alberi diversi in esecuzioni diverse (dipende dall'ordine con cui scegli pick(Unvisited) e dai tempi). Con SHOUT non puoi prevedere quale spanning tree ottieni.

In generale, **DFT può costruire alberi con diametro pessimo** (molto grande), e per broadcasting sarebbe ideale avere un albero con **diametro piccolo**. Una strategia “ideale” è costruire un BFS tree radicato in un centro del grafo, ma trovare il centro e costruire BFS tree può essere costoso.

Mini-checklist da esame (cose da saper dire al volo)

- Definizione di entry e perché determina il **parent**
- Perché ForwardToken su nodo già visited $\Rightarrow$ BackEdgeToken
- Perché “rimuovendo le back-edges” ottieni un albero
- Complessità base:

$$2m$$

messaggi e

$$2m$$

tempo; LB tempo

$$\geq n - 1$$

- Ottimizzazione Visited/Ack: tempo

$$O(n)$$

e messaggi

$$O(m)$$

con costante $\approx 4m$